

计算机操作系统讲义

第1章

计算机操作系统概述

本章是对计算机操作系统的一个概述，主要介绍有关计算机操作系统的一些基本概念，操作系统的形成及发展过程，操作系统的分类，操作系统的功能及几种常用操作系统（DOS、WINDOWS，UNIX、LINUX）的历史简介。

1.1 什么是操作系统

1、 计算机系统的组成

计算机系统由计算机硬件和计算机软件两部分组成。
计算机硬件包括中央处理单元 CPU（central processing unit）、存储器（memory, store, storage）、外围设备（peripheral devices）。
计算机软件包括系统软件(system software)、应用软件(application software)

2、 操作系统的定义

计算机操作系统是方便用户、管理和控制计算机软硬件资源的系统软件（或程序集合）。

3、 常用的操作系统

Msdos、OS/2、Windows、Unix、Linux 等。

1.1.1 操作系统在软件层次中的地位

1、 操作系统在软件层次中的地位

操作系统是一种系统软件，系统软件还包括语言处理软件、例行服务程序等。软件的分

表 1-1 软件分类

软件	系统软件	操作系统
		语言处理系统
		例行服务程序
	应用软件	用户程序系统

注：1、语言处理程序包括各种语言的编译程序、解释程序、汇编程序。
2、 例行服务程序包括库管理程序、链接程序、编辑程序及系统监控程序等。

3、 操作系统在软硬件层次中的地位

操作系统是计算机硬件的扩充，“裸机”+操作系统=虚拟机
一个裸机在每加上一层软件后，就变成了功能更强的虚拟机。操作系统与软硬件的层次关系如图 1-1。



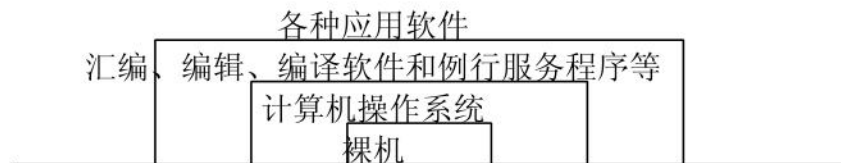


图 1-1 操作系统和硬件的层次图

4、从不同的角度理解操作系统

①从人机交互的角度

操作系统是用户与计算机的接口，具有友好方便的用户接口界面。

②从管理者的角度

操作系统是所管理资源的程序扩充。操作系统的工作就是分门别类地管理，并详细记录资源的使用情况，再按一定的策略对资源进行调度分配，为用户服务。

③从计算机系统结构的角

操作系统是一种层次化、模块化结构的程序集合。

④从本课程的知识结构的角

操作系统是计算机技术和管理技术的结合。

1.1.2 操作系统的设计观点

操作系统的设计观点包括用户观点和资源两方面。即一方面要面向用户服务；另一方面还要充分利用计算机的软硬件资源。

1.2 操作系统的形成和五大类型

计算机操作系统是随着计算机硬件的发展而逐步形成和发展起来的，早期的计算机操作系统（60 年代以前）是以串行批处理的方式进行工作的；60 年代中后期由于中断（interrupt），通道（channel）技术的出现，使得并行处理成为可能，这为分时操作系统和实时操作系统奠定了基础。80 年代后，由于计算机网络的出现，网络操作系统及分布式操作系统开始出现。

1.2.1 计算机操作系统的形成

第一代计算机（1945—1955）：电子管和手工操作

这一代计算机由主机（运算控制部件、内存）、输入设备（纸带输入机、卡片阅读机）输出设备（打印机）构成，其特点是运算速度慢，体积庞大。这一阶段是手工操作，无操作系统。

第二代计算机（1955—1965）：晶体管和批处理系统

随着计算机处理速度的提高（每秒几千次提高到每秒几百万次），为了提高计算机的自动化程度，人们研制了监督程序，由该程序自动依次处理一系列任务，于是进入了“单道批处理阶段”。该阶段可分为以下三个时期。

1. 联机批处理系统

（1）工作方式（如图）



(2) 系统的优点

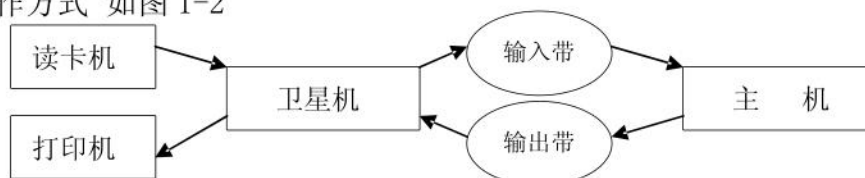
提高了计算机的自动化程度，减少了人工干预。

(3) 系统的缺点

快速的 CPU 和慢速的 I/O 设备之间产生矛盾，造成 CPU 资源的浪费。

2. 脱机批处理系统

(1) 工作方式 如图 1-2



图案 1-2 脱机批处理系统工作方式示意图

(2) 系统优点

缓解了 CPU 与 I/O 设备之间的矛盾，提高了 CPU 的资源利用率。

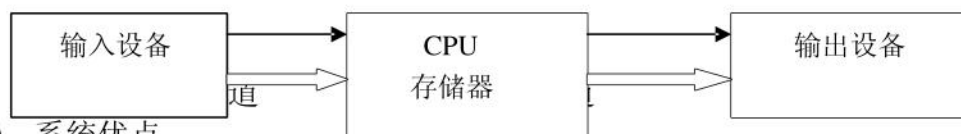
(3) 系统缺点

CPU 与外围计算机完全隔离，可能造成系统“死机”。

3. 执行系统

(1) 工作方式

系统的构架同于联机批处理方式，但通道和中断技术的引进，使 CPU 和 I/O 设备，CPU 和通道的并行操作成为现实。(如图)



(2) 系统优点

克服了联机批处理系统和脱机批处理系统的缺点

(3) 系统缺点

作业的处理仍然是串行的。

第三代计算机（1965—1980）：集成电路芯片和多道程序设计技术

这一阶段操作系统初步形成并逐步完善，形成了三种基本的操作系统：多道批处理系统，分时操作系统和实时操作系统。

第四代计算机（1980—1990）：大规模集成电路芯片和传统操作系统。

操作系统向着改变用户界面，方便用户的方向发展。

第五代计算机（1990—）超大规模集成电路和网络及分步式操作系统。

以上的历史总结见表 1-2

1.2.2 操作系统的五大类型

现有的计算机操作系统分为

批处理操作系统 (batch processing operating system);

分时操作系统 (time sharing operating system);

实时操作系统 (real time operating system);

网络操作系统 (network operating system);

分布式操作系统 (distributed operating system);

1. 批处理操作系统

(1) 多道程序设计技术的概念

多道程序设计技术就是在系统（内存）中同时存放并运行几道相互独立的程序。它是一种宏观上并行，微观上串行的运行方式，内存中的分区如图 1-3

操作系统
程序 A（内存分区）
程序 B（内存分区）
程序 C（内存分区）

图 1-3 一个多道程序系统在内存中的分区示意图

(2) 多道批处理操作系统系统

多道批处理操作系统系统=批处理系统+多道程序设计技术；
其优点是成批处理作业，提高了作业吞吐量，多道程序运行，提高了资源的利用率。缺点是缺乏交互性。主要运用在大计算量的科学计算上。

(3) 几个概念

- 多道程序系统 (multi-programming system)
- 多重处理系统 (multi-processing system) :多 CPU 系统
- 多用户系统 (multi-users system): 它一定是多道程序系统，但反之不然。

2. 分时操作系统

- (1) 分时技术：将 CPU 的时间分成很短的时间片（几十毫秒-几百毫秒）轮流为每个用户工作，采用这种技术的操作系统称为分时操作系统。
- (2) 优缺点：优点是控制简单，系统稳定性好； 系统交互性好。
缺点是用户的优先级不易控制。

3. 实时操作系统

- (1) 实时操作系统要求计算机对外来信息能以足够快的速度进行处理，并在被控对象允许的时间范围内作出快速响应。主要应用在实时处理之中。
- (2) 优点是响应速度快，缺点是交互能力差。

4. 网络操作系统

- (1) 网络操作系统包括网络管理、通信、资源共享、系统安全和多种网络应用服务。
- (2) 多用户系统和网络系统的比较（表 1-3）

	多用户系统	网络系统
CPU	共用一个或几个	多个处理器
共享资源	共享主存	共享服务器
终端工作	分时	网址通信链接
客户端工作	不能独立工作	客户机能独立工作
操作系统	集中管理	有自己的操作系统

表 1-3 多用户系统与网络系统的比较说明

5. 分布式操作系统

(1) 分布式操作系统的特征

- (a) 需要一个全局的文件系统;
- (b) 所有的 CPU 上运行同样的内核, 统一的管理和控制机构, 进行优化的协调工作;
- (c) 有一个单一的、全局的进程通信机制。提供选择优化本地和远地的资源利用。
- (d) 有全局的保护机制。

(2) 目前还仅仅是一个发展方向, 无现成的产品推出。

1.3 操作系统的五大功能

操作系统的五大功能分别为:

- (1) 作业管理 (job management);
- (2) 文件管理 (file management);
- (3) 存储管理 (store, storage, main storage management);
- (4) 设备管理 (devices management);
- (5) 进程管理 (process management);

1.3.1 作业管理

作业管理包括任务 (task) 管理, 界面 (interface) 管理, 人机交互的图形界面 (icon), 联机控制 (on-line), 脱机控制 (off-line), 假脱机 (spooler) 控制和作业调度及调度算法。

1.3.2 文件管理

文件管理又称为信息 (message) 管理, 它是对计算机的软件资源的管理, 其中包括文件的存储、检索、共享、保护等的方法、技术及算法。

1.3.3 存储管理

主要涉及内存空间充分利用的技术、多道、多重处理及内存的分配、保护和扩充。

1.3.4 设备管理

它是对计算机硬件的管理, 主要包括对 I/O 设备的分配、启动、完成及回收, 主要的技术有假脱机技术 (SPooling)、队列及缓冲技术、设备驱动程序等。

1.3.5 进程管理

简单的讲, 进程就是一段正在运行的程序, 进程管理的主要内容包括: 进程调度及调度算法、进程间的通信、进程的死锁及解决的办法。由于进程管理和 CPU 的管理密切相关, 因此本教材将其并为一章。

1.4 操作系统版本历史简介

1.4.1 DOS 操作系统简史

见表 1-4

1.4.2 Windows 操作系统简史

见表 1-5

1.4.3 UNIX 操作系统简史

见表 1-6

UNIX 操作系统的变种见表 1-7

1.4.4 Linux 操作系统简史

见表 1-8

1.5 表征操作系统的属性

1. 响应比，响应系数（Response Ratio）

(1) R_p =作业响应时间/运行时间

影响 R_p 的因素有：CPU 速度，内外存对换，I/O 调度，用户数，时间片，事件优先权等

(3) R_p 与设备利用率的关系

如下表 1-9

操作系统类型	响应系数	设备利用率
批处理	最大，>100	最高
分时交互	中等，<100	中
实时	最小，<5	最低

表 1-9 操作系统响应系数与设备利用率的比较

2. 并发性（concurrent）也叫“共行性”

并发性的特征有如下几点：

(1) 多个作业并发执行或一个用户作业的多个程序段间并发执行：

(2) 多个 I/O 设备间并发工作：

3. 信息的共享、保密与保护

操作系统中常见的加密方法有三种，分别为

(1) 给用户设置登录口令：

(2) 给文件加权限：

(3) 给文件加密。

4. 可扩充性、可移植性、可读性、可“生成”（generation）性

5. 可测试性

(1) α 测试与 β 测试

(2) 常用的测试指标

(a) MIPS（million instruction per second）——每秒百万条指令

(b) TPS（transactions per second）——每秒处理的事务数

一般地，(a) 用着测试计算机的数据处理能力，(b) 用着测试计算机的 I/O 处理能力。

6. 安全可靠

7. “行业评测性能比较”

8. 几种常用的操作系统特点比较

见表 1-10

系统名称	用户数	任务特点	发展特点	共性
DOS	单用户	单任务	命令控制	树型目录结构 I/O 重定向及
Windows 3.x	单用户	多任务（协作式）	事件驱动	

Windows 95	单用户	多任务(半抢先式)	PnP, 32 位、16 位	管道功能
Windows NT	多个 C/S 模式	多任务、多线程	增强网络应用	
UNIX 或 Linux	多用户	多任务(抢占式)	分时(固定时间片)	

表 1-10 几种常用操作系统特点比较

1.6 操作系统的“生成”、设置和配置概念

1.6.1 “生成”(system generation)、配置(configuration)和设置(setup)

1. 操作系统的安装

各种操作系统均提供一个安装程序为操作系统的安装提供方便,如 DOS、WINDOWS 的 SETUP.EXE,UNIX 的 N1 盘。安装的方式一般有如下几种:

- (1) 最小安装
- (2) 典型安装
- (3) 完全安装
- (4) 自定义安装

2. 操作系统的配置和设置

配置和设置的目的是为了形成更适合用户的操作系统环境。

3. 操作系统的“生成”

所谓操作系统的“生成”(generation)是指将操作系统安装在计算机上,并对操作系统作适当配置而形成操作系统内核(kernel)的过程。

1.6.2 操作系统引导

1. 几个基本概念

- (1) 基本输入输出系统---BIOS (basic input output system),它是与计算机硬件打交道的低层代码,由计算机厂商提供,一般固化在只读存储器 ROM 中。
- (2) 小型计算机系统接口---SCSI (small computer system interface),它是计算机外部设备的一种标准接口方式。
- (3) 内存及地址---内存一般指 RAM (random access memory),内存中每一字节都赋以一个编码,这个编码称为地址(address),它是一个 16 进制的数字

2. DOS、Windows 的启动过程

(1) DOS 的构成

DOS 由 BIOS、核心和 Shell 三个层次组成,BIOS 是底层设备驱动程序的集合,BIOS 使得 DOS 可以与具体的机器硬件相分离。DOS 的隐含文件 io.sys 提供 DOS 与 BIOS 的调用接口。而核心放在文件 msdos.sys 中,它负责进程管理、存储管理、与文件管理、解释系统调用并执行。Shell 即为命令解释程序,存放在文件 command.com 中,它负责键盘命令的解释及执行。

(2) DOS、Windows 的启动过程

(a) 系统加电

系统加电时，主板的控制芯片向 CPU 发一个 RESET 信号，待电源稳定后，撤去 RESET，CPU 从 ROM 中的地址 0xFFFF0 开始执行指令。

(b) 加电自检 (POST)

主要是检查系统中的一些关键设备（内存、显示卡）是否存在。

(c) 检查各种设备的 BIOS

主要是检查及初始化各种设备（显卡、IDE 卡、SCSI 卡等），设备初始化后，系统显示设备的状况。

(d) CPU 及 RAM 检查

检查 CPU 的类型、时钟频率及 RAM 的校验。

(e) 检测系统中的其他 I/O 设备

主要有硬盘、CD-ROM、软驱、串口、并口等

(f) 检测系统中的即插即用设备

检测 PnP 设备并为其分配资源（中断、DMA 通道及端口地址）

(g) 系统更新 ESCD 中的数据

ESCD—extended system configuration data，它是系统 BIOS 与 DOS 交换硬件配置的手段，它存放在 CMOS 中。

(h) DOS、Windows 启动

系统 BIOS 从启动盘的零磁道读入主引导记录，主引导记录将 io.sys 及 msdos.sys 调入内存；读入 config.sys 来作系统最后的配置；DOS 本身读入 Command.com，Command.com 执行自动批处理文件 autoexec.bat，系统处于待命状态。

1.6.2 系统管理员

系统管理员的工作内容如下

1. 系统管理员任务
2. 一般日常工作
3. 阶段性工作
4. 维修常识

1.7 重点小节

1. 计算机操作系统是方便用户、管理和控制计算机软硬件资源的系统软件（或程序集合）。
2. 操作系统有五大类型——批处理、分时、实时、网络和分步式操作系统。
3. 操作系统的设计有用户观点和系统资源管理的观点，面对用户的观点主要是为用户提供方便、安全的工作环境及良好的用户界面；系统资源管理的观点则是对计算机的各种资源进行记录、分配、调度及回收。
4. 操作系统是用户和计算机的接口。一般用户采用命令的方式与计算机进行交互；编程人员则用系统调用的方式控制计算机。
5. 操作系统采用多道程序设计、分时、并行、并发等技术来提高 CPU 处理多个任务的能力，以提高 CPU 的利用率。
6. 采用通道、中断、缓冲、队列等技术来解决高速 CPU 和慢速 I/O 设备之间的矛盾。

第2章

第3章 作业管理

本章包含两个方面的内容，一为作业的组织及调度；另一个内容是用户界面的任务和功能。

2.1 作业管理的任务和功能

2.1.1 概论和术语

1. 作业 (job)

(1) 用户角度

我们把一次应用业务处理过程中，从输入开始到输出结束，用户要求计算机所做的有关该次业务处理的全部工作称为一个作业。

如图所示的一般编程过程可以认为是作业的一个例子。

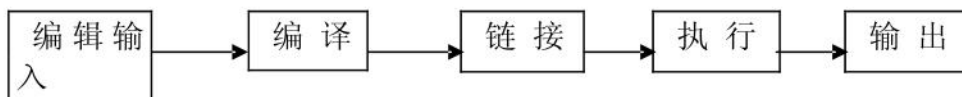


图 2-a 一般编程过程

(2) 系统角度

从计算机系统的角度看，作业是一个比程序更广的概念，它由程序、数据和作业说明书三部分组成。

系统通过作业说明书控制文件形式的程序和数据，使之操作和执行。在批处理系统中，作业是抢占内存的基本单位。也就是说，批处理系统是以作业为单位把程序和数据调入内存以便执行的。

2、作业步 (job steps)

用户把要求计算机系统做的一项相对独立的工作叫做一个作业步。例如图 2-a 中，编译、链接、执行就是作业步。

作业由不同的顺序相连的作业步组成，一般来说，每一个作业产生下一个作业步的输入文件。

3、作业同步 (job synchronization)

所谓作业同步是指在一个作业中，下一个作业步能否执行下去，取决于前一个作业步是否成功完成。

4、作业流 (job stream)

在系统控制下，将一批作业依此输入到后援存储器中等待运行，这样就形成了一个作业流。

5、作业类型 (job type)

用户作业分为两大类：分别为批量型作业和终端型作业

(1) 批量型作业可分为两种，利用作业说明书实行自动控制方式的作业称为脱机作业；利用控制台键盘操作命令直接控制的作业称为联机作业。

(2) 终端型作业又称为交互型或会话型作业，通常在分时操作系统环境下运行，用户在终端上利用键盘命令控制和监督作业的运行，而系统把作业运行的情

况和结果也及时反馈在用户终端上。在大型的操作系统中，常把终端用户作业称为“前台”作业，把批量型作业称为“后台”作业。

2. 1. 2 作业管理的任务

作业管理的任务是为用户建立作业，组织调用系统内部资源执行，并在完成任务后将其撤消。

1、 作业的建立过程

一个作业的建立过程一般是作业的输入和系统建立起作业控制块（JCB—job control block）两个子过程。如图 2-b 所示



图 2-b 作业建立过程

2、 作业输入方式

作业输入方式可分为五种，即联机输入方式、脱机输入方式、直接耦合方式、SPooling（simultaneous peripheral operations online）系统和网络输入方式。

（1） 联机输入方式

联机输入方式大多用于交互式系统中，用户和系统通过交互会话方式输入作业。近年来由于多媒体技术（Multi-Media）的发展，逐步形成了手写输入、语音输入、光电输入等输入方式。

（2） 脱机输入方式

脱机输入方式又称为预处理方式，脱机输入方式利用低档个人计算机进行输入处理。在低档个人机上，用户通过联机方式把作业首先输入到后援存储器，如磁盘或磁带上；然后，用户把装有输入数据的后援存储器拿到主机的高速外围设备上和主机相连，从而在较短的时间内完成作业的输入工作。

脱机输入方式的优点是解决了作业的快进快出，相应提高了 CPU 的利用率。但其缺点也是明显的，主要有：

（a） 需要人工干预，这使得作业的响应时间和系统的出错率受人的因素影响。

（b） 增加了作业周转时间，在这种方式下，必须将作业的全部输入输出信息放入磁盘或磁带后，才能拿到主机上，这样增加了作业的周转时间。

（c） 不易实现优先级调度算法，这种方式的作业顺序是事先安排好送入磁带或磁盘上的，只有等前一批作业处理完成，下一批作业才能开始处理。

（3） 直接耦合方式

它用一个容量的共用存储器，把多台用作输入的低档机、共用存储器和主机固定连接起来，保留了脱机输入方式的优点，又克服了该方式需人工干预的缺点。工作方式如图 2 -c



图 2—c 直接耦合输入方式

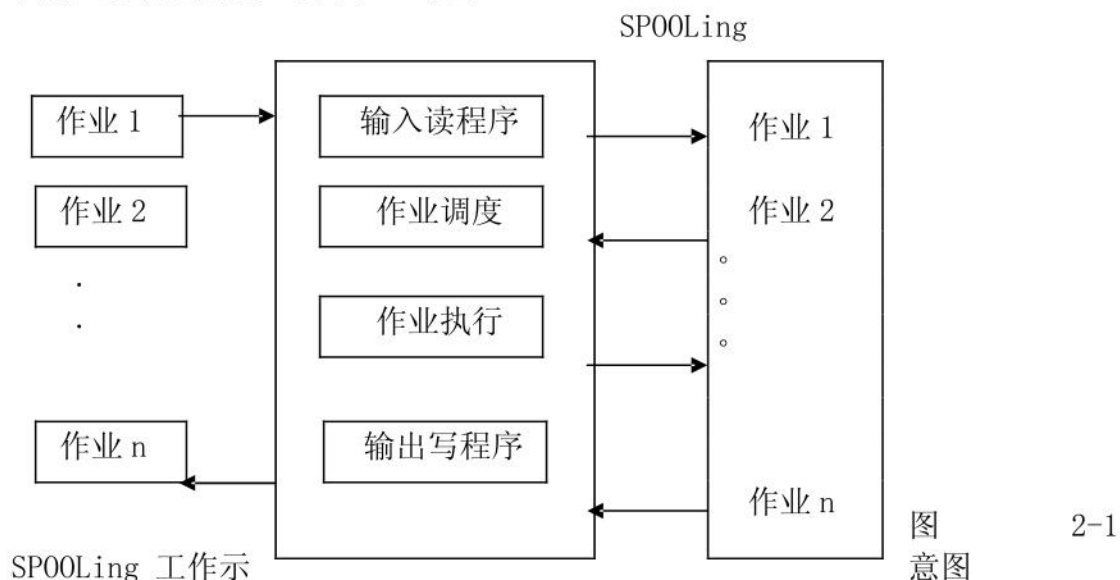
(4) 假脱机输入方式

(a) 假脱机技术(SPOOLing 或 SPOOLer)

SPOOLing 技术实际上是一种外围设备同时联机操作技术(simultaneous peripheral operation on-line)的缩写。又称为排队转储技术。

(b) 工作原理

SPOOLing 系统既不同于脱机方式，也不同于直接耦合方式。它在输入和输出之间增加了“输入井”和“输出井”的排队转储环节，以消除用户的“联机”等待时间。在系统输入模块收到作业输入请求信号后，输入管理模块中的读过程负责将信息从输入装置中读入输入井缓冲区。当缓冲区满时，由写过程将信息从缓冲区写到外存的输入井中，读过程和写过程反复循环，直到一个作业输入完毕。当读过程读到一个硬件结束标志之后，系统再次驱动写过程把最后一批信息写入外存输入井并调用中断处理程序结束该次输入。然后，系统为该作业建立作业控制块，从而使输入井中的作业进入作业等待队列，等待作业调度程序选中后进入内存运行。系统在管理输入井过程中可以“不断”读入输入的作业，直到输入结束或输入井满而暂停。如图 2-1 所示。



(5) 网络输入方式

当用户需要从计算机网络中将一台计算机的信息要求传送到联网的另一台主

机上进行浏览（Browser）操作或执行下载（Download）等任务要求时，就构成网络输入方式。主要涉及网络通信技术。

2.1.3 作业管理的功能

- (1) 分门别类地组织控制作业的运行，解决作业的输入输出问题；
- (2) 了解和申请机器资源；
- (3) 系统工作状态的记录、跟踪、监控及调试；
- (4) 提供用户或程序员在程序工作模式下的服务功能。

2.1.4 操作系统是用户和计算机之间的接口

操作系统作为用户提供两种接口，其中一类是为一般用户提供的操作命令接口，另一类为提供给编程人员的系统调用（system call）接口。

1、 命令接口

命令接口包括联机的键盘操作命令和脱机的作业控制命令或称作业控制语言。一般的操作系统仅提供其中之一，仅仅在一些大型机或巨型机的操作系统才提供脱机的作业控制命令。

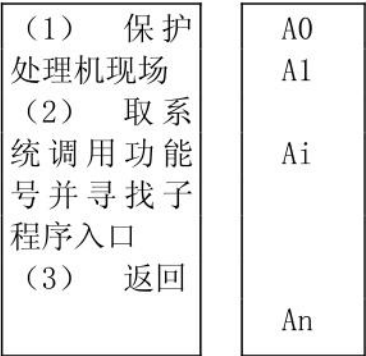
2、 系统调用

(1) 系统调用的含义

系统调用是操作系统为编程人员提供的接口，各种操作系统的核心中都设计有一组一组的用于实现各种系统功能的子程序作为机器指令的扩充。系统将这些子程序“开放”给用户，方便用户可靠地调用系统有关的资源，而用户不必从头熟悉或重新编写子程序。每当用户在程序中需要操作系统提供某种服务是，便可利用一条条相应的系统调用命令，去调用所需的系统过程。

(2) 系统调用的处理过程

为了保障操作系统的安全性，使得操作系统的程序不被用户程序破坏，一般操作系统都不允许用户程序访问操作系统的系统程序及数据。在机器的指令系统中有一条称为“访管”的指令，编程人员可以利用这条指令来访问操作系统并向它提出要求。访管指令有“参数区”、“参数”和“操作数”组成。“操作数”用来表示请求操作系统所要干的工作，并说明是否要有参数区和具体参数。参数或参数区的首址通常约定放在某个通用寄存器中，但 CPU 执行到“访管”指令时，将“访管”指令存入主存中的约定单元，然后产生“访管”中断，引出操作系统来处理“访管”中的具体要求。处理过程如图 2-d



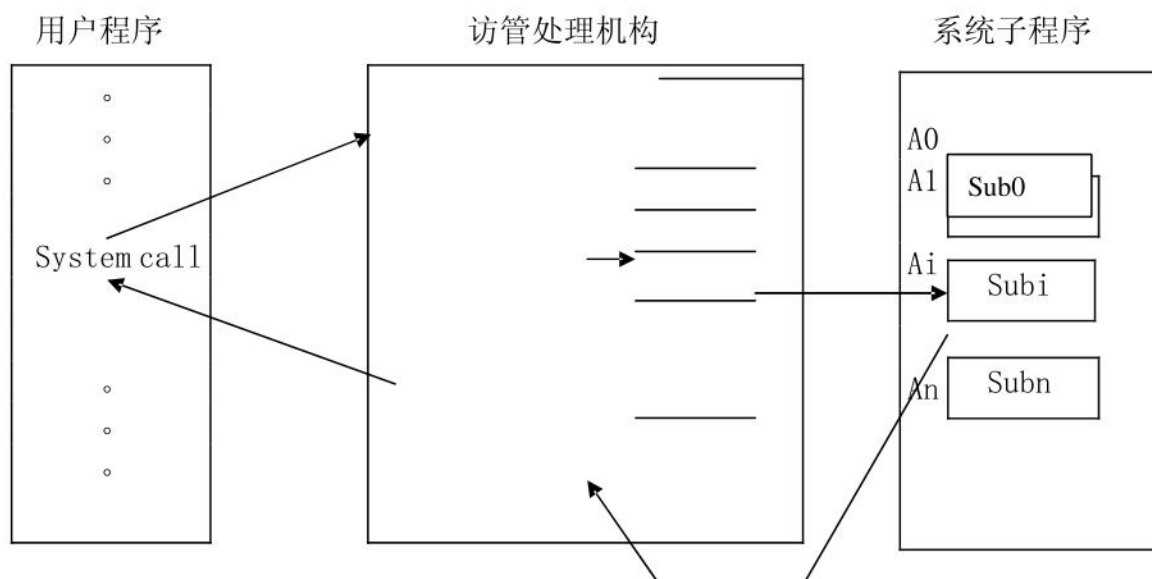


图 2-d 系统调用的处理过程

(3) 名词解释

访管指令—把由于系统调用引起的处理机中断的指令称为访管指令。

广义指令—利用“访管”指令来定义的指令称为广义指令。是通过执行相应的程序模块来实现的，它是机器指令的扩充。

(4) 广义指令与宏指令的区别

宏指令 (macro-instruction) 是源程序中的某一宏 (macro) 定义的缩写。它是作为用户语言被处理，作为开式子程序展开到目标程序中，往往可由用户自行定义。

2.2 用户界面的任务和功能

2.2.1 用户界面的发展

1. 第一代用户界面

第一代用户界面为一维界面，主要有命令行界面和编程人员在程序中的系统调用，如 DOS 及 UNIX 均采用此种界面方式。

2. 第二代用户界面

第二代用户界面为二维界面又称为图形界面，以窗口 (windows), 图标 (icon)、菜单 (menu) 为典型特征，由 APPLE 公司开创，以 Microsoft 公司的 MS-Windows 为里程碑，在 UNIX 系统下有 X-window。

3. 第三代用户界面

第三代用户界面为三维界面，又称为虚拟现实 (virtual reality)，如三维动画设计、可视电话及网络视频会议等。

2.2.2 界面管理的任务

作为面向最终用户的“作业”管理来看，用户界面已经成为计算机系统的一个重要组成部分，是计算机科学与心理学、图形艺术和人类学的交叉研究领域。寻求最佳的人机通信方式已是多媒体、虚拟现实和科学计算、可视化等技术所追求的

目标，也是界面管理的、最终任务。

2. 2. 3 界面管理的功能

1. 实现高效的人机通信

将人的听、说、手和眼等协同动作，采用多通道、以自然的方式交互，可以实现高效的人机通信。

2. 改善计算机的可用性、可学性和有效性

要求计算机能理解人的要求，甚至能纠正人的错误，智能化的界面是一个重要方向。

3. 支持三维及多媒体技术

需要有语音、图像和动画等多媒体支持。

4. 为广大用户提供适应不同应用的众多界面构造工具及语言

现有的 GUI 的开发工具有 Visual Basic、Visual C++、Visual FoxPro、X-window 等。

2. 2. 4 用户界面研究

1. WIMP 界面

WIMP 界面是指窗口 (windows)、图符 (icons)、菜单 (menu)、指点装置 (pointing devices)，是目前个人计算机流行的用户界面。

2. WYSIWYG 技术

WYSIWYG—what you see is what you get 即为“所见即所得”

3. 多通道

多同道—multi-modal user interface, MMI

4. 人机询问技术

人机询问技术—man-machine interrogation technique, MMIT

人机交互技术的新发展，使机器能象人一样地进行“交谈”。

5. Post-WIMP 时代（后 WIMP 时代）

同时操作多个外设，例如多个传感器、多个通道、多个人同时使用以至能识别人的双手及人的形体语言的多模式识别。

6. X-window

X-window 是 1986 年由美国 MIT 的雅典娜工程开发小组为了提供一个独立于硬件的用户图形界面而开发成功的，广为流行的是 1988 年 3 月发行的 X11.2 版本。尤其与 UNIX 操作系统结合，可运行于 IBM-PC 到各种大中型机以及巨型机上，它又可运行在 X 终端上。同时它也给程序员提供了工具箱 (toolkit) 和事件驱动型编程模型。

6. 虚拟现实

虚拟现实 (virtual reality, VR) 是用计算机技术来生成一个逼真的三维视觉、听觉、触觉、嗅觉等感觉世界，让用户可以从自己的视点出发，利用自然的技能和某些设备对这一生成的虚拟世界进行浏览和交互考察。实际应用的虚拟现实系统大体可分为：

- (1) 桌面虚拟现实系统
- (2) 沉浸虚拟现实系统
- (3) 分布式虚拟现实系统

(4) 增强现实

2.3 用户界面的设计特点

1. 计算机不断适应人的需求，以方便用户为目标。
2. 用户界面设计是计算机科学与心理学、图形艺术和人类学的结合。
3. 多窗口系统一般都采用事件驱动（event drive）的控制方式。

2.4 作业调度

2.4.1 作业调度功能

- (1) 采用作业控制块（JCB）表格，记录系统中各作业工作状况；
- (2) 根据选定的调度算法，从后备作业中选出一部分（多道情况）或一个作业投入运行；
- (3) 为被选中的作业做好运行前的准备，包括选择系统相应的“进程”执行单元以及为这些“进程”分配系统资源，首先判断用户的资源要求是否能够满足；
- (4) 作业处理后的善后处理工作，例如，回收资源和记帐等工作。

2.4.2 作业说明书内容举例

作业说明书主要包括三方面的内容，即作业的基本描述、作业控制描述和资源要求。如图 2-e

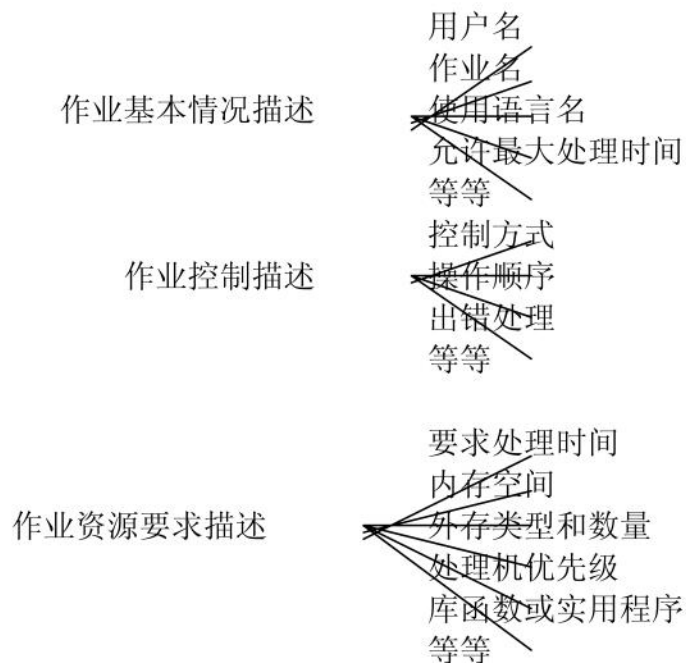


图 2-e 作业说明书的主要内容

2.4.3 作业调度算法

1. 先来先服务（first come first serve, FCFS）

按作业进入输入井的先后次序安排。优点是实现简单，用先进先出（first input first output, FIFO）队列顺序工作，对相同的或均衡的作业较为合理，缺点是不利于运行时间短的作业。

2. 最短作业优先法 (shortest job first, SJF)

(1) 概念

(a) 作业 i 的周转时间 T_i = 作业完成时间 - 作业提交时间

(b) 平均周转时间 $T = 1/n \sum T_i$

(c) 平均带权周转时间 $W = 1/n \sum T_i / t_i$

t_i 为作业的实际运行时间。

2. 最短作业优先法

最短作业优先法也就是选 t_i 值小的优先，也就是只考虑运行时间。优点是短作业得到了优先执行，提高了系统的效率。缺点是当作业不断进入时，长的作业有可能长时间排不上队

3. 最高响应比优先法 (highest response-ratio next, HRN)

最高响应比优先法 (HRN) 是对 FCFS 方式和 SJF 方式的一种综合平衡。HRN 调度策略调度同时考虑每个作业的等待时间长短和估计需要的执行时间长短，从中选出响应比最高的作业投入执行。

响应比或称响应系数比 R 定义下：

$$R = (W + T) / T = 1 + W/T$$

其中 T 为该作业估计需要的执行时间， W 为作业在后备状态队列中的等待时间。其优点是同时具有 FCFS 算法及 SJF 算法的优点，缺点是实现复杂，每次调度前要对所有作业扫描一遍，比较以后再调度。

4. 定时轮转法

按时间片轮转，可分为短时间的固定时间片（如，UNIX 操作系统时间片为几毫秒至几十毫秒）和长时间的不固定时间片（如：Windows 操作系统的抢占式多任务方式）。

5. 优先数法

按优先数排队次序工作。分静态和动态：静态是在排队前计算优先数，动态是在调度时计算优先数。又可分为用户给定优先数（反映用户要求）和系统给定的优先数，例如，系统给定前台和后台工作的优先级，一般前台优先。

6. 事件驱动法

MS-Windows 采用的任务驱动方式，采用不固定的时间片分配来完成多任务。每当发生一些事件 (event) 就进入相应的事件调度程序。系统通过事件驱动程序执行任务。

7. 各种不同类型作业搭配调度算法

一个操作系统的作业调度往往是综合性的，作业调度的原则主要要考虑的因素有 CPU 的运行时间、对内存的要求、对外设的要求，对 I/O 吞吐量的要求及等待时间等等。例如，JUNE OS，其优先数 (P)、等待时间 (t_w)、要求运行时间 (t_r) 以及输出量 (E) 的调度算法如下：

$$P = t_w^2 - t_r - 16E$$

2.5 操作系统常用命令分类简介

2.5.1 操作系统是人机交互的接口

计算机操作命令可分为联机命令、Shell 程序语言命令和脱机作业控制语言命令。

1. 联机命令

联机命令用于人机交互系统中，联机命令接口包括联机命令、终端处理程序和命令解释程序。

2. Shell 程序命令

Shell 程序命令是利用现有计算机语言功能，有会话式和命令文件两种输入方式，提供给用户的一种综合应用。

3. 脱机作业控制语言命令

在批处理操作系统中，用户利用作业控制语言（job control language、JCL）命令，写成作业说明书提交给系统管理员输入系统，由系统按照作业说明书上的命令，逐条解释执行。用户一旦把作业给系统后，便失去了自己干预作业交互的能力。这种命令接口可称为脱机命令接口。

4. UNIX 操作系统的命令分类

- (1) 有关文件和目录的管理
- (2) 有关文件的处理和编辑
- (3) 有关文件系统的装卸
- (4) 有关系统管理和工作状态
- (5) 有关通讯和个人通讯
- (6) 有关硬件设备处理
- (7) 有关程序开发
- (8) 其他杂项

2.5.2 常用命令对比

计算机操作系统命令的常用命令对比见表 2-1

2.5.3 UNIX, Linux 联机命令手册索引

usr/man 目录下分类：

- (1) man 1 系统常用命令
- (2) man 2 系统调用
- (3) man 3 文本调用
- (4) man 4 特殊文件（设备管理）
- (5) man 5 文件格式
- (6) man 6 游戏说明

- (7) man 7 杂项（UNIX）
宏命令包（Linux）
- (8) man 8 系统管理与维护工具
- (9) man 9 核心例程（Linux）

2. 6 Shell 命令解释和控制语言

Shell (外壳程序) 语言最早是由 UNIX 操作系统提供给用户使用的命令解释程序集合。在 1978 年由 Bourne 开发而成, 称为 B_Shell。1983 年 joy 等人在第六版基础上又开发成 C_Shell, 最大的特点是采用了 C 语言的语法词法结构。主要用在 BSD 系列的 UNIX 系统中。MS-DOS 在 5.0 的版本以后, 也提供了类似的外壳语言。

2. 6. 1 DOS 操作系统 Shell 语言

DOS 操作系统的 Shell 语言由 DOS 命令集和及相应的控制命令 (批处理子命令) 组成, DOS 提供了 7 条控制命令, 分别为 ECHO (显示开关)、FOR (重复执行)、GOTO (控制转向)、IF (条件执行)、PAUSE (暂定系统运行)、REM (显示注释信息) 和 SHIFT (移位替换参数) 等。AUTOEXEC.BAT 是批处理文件的一个典型例子。

2. 6. 2 作业管理和进程管理的类比说明

作业调度又称为高级处理机管理或宏处理机管理。其主要工作是确定计算机系统中哪些作业以至哪个作业将获得 CPU 的服务工作; 而进程调度又称为低级处理机管理或微处理机管理, 其主要工作是确定计算机系统中哪些作业的进程将获得 CPU 的服务工作。

2. 7 重点小结

1. 熟悉人机界面的发展特点。
2. 利用 一种操作系统 (DOS、Windows、UNIX 或 Linux), 来体会操作系统的功能。
3. 掌握系统调用的原理及基本的系统调用的使用。
4. 通过上机练习掌握作业调度算法的模拟编程。

第三章 文件管理

3. 1 文件管理的任务和功能

3. 1. 1 文件管理的任务和功能

文件管理的任务是把存储、检索、共享和保护文件的手段提供给操作系统本身和用户, 以达到方便用户和提高资源利用率的目的。

操作系统的文件管理主要是通过数据结构的表格 (在 UNIX 或 Linux 操作系统中称为块) 来进行, 如: DOS 操作系统中的文件目录表 (file directory table, FDT) 和文件分配表 (file allocate table, FAT)。

1. DOS 操作系统的文件管理简介

(1) DOS 文件系统是层次结构的文件系统, 允许用户在任何驱动器的任何子目录上创建、打开、关闭和删除文件, 以及在文件的任意字节偏移处读写任意长度的记录。

(2) 文件的逻辑组织

磁盘文件被组织成层次结构, 由一个逻辑卷、一个根目录、零到多个子目录和文件。

(3) DOS 磁盘映象包含引导程序、文件分配表、根磁盘目录和文件区。

逻辑扇区 0 用来存放引导程序，逻辑扇区 1 用来存放文件分配表，可能文件分配表(FAT)占始址为 1 的几个连续的扇区，随后是磁盘根目录(FDT)，也可能占几个连续的扇区，剩下的是文件区，用于存放通常意义下的数据文件或目录文件。

2. MS—DOS 和 Windows95/98 中 FAT32 和 FAT（16）的分区及簇大小比较
分区及簇大小比较见表 3—1。

FAT32		FAT（16）	
分区大小	簇大小	分区大小	簇大小
<260MB	512bytes	<127MB	2KB
260MB—8GB	4KB	128MB—255MB	4KB
8GB—16GB	8KB	256MB—511MB	8KB
16GB—32GB	16KB	512MB—1023MB	16KB
>32GB	32KB	1GB—2GB	32KB

表 3-1 FAT32 和 FAT（16）的分区及簇大小比较表

(1) FAT（16）
它是 MS—DOS 和 Windows 95 操作系统中最常见的磁盘分区格式。它采用 16 位的文件分配表，最大能支持 2GB 的硬盘。
其缺点是磁盘利用率低。由于磁盘文件的分配是以簇为单位，FAT16 支持的分区越大，磁盘上每个簇的容量也越大，造成的浪费也越大。

(2) FAT32
它是微软公司从 Windows 97 后推出的一种新的磁盘分区格式，采用 32 位的文件分配表，增强了对磁盘的管理能力。
其优点是：

- (a) 突破了 FAT16 对一个分区最大为 2G 的限制；
- (b) 在不超过 8GB 的分区中，FAT32 分区格式的每个簇的容量都固定在 4KB，可大大地减少磁盘空间的浪费。

其缺点是：
(a) DOS 不支持这种分区，DOS 文件无法运行；
(b) 磁盘的运行速度减慢。

3. 数据结构说明

数据结构一般包括堆栈(stack)、队列(queue)、表格(table)、树型(tree)、和图论(graphic)、等，操作系统的文件管理大量使用表格这种数据结构。

3.1.2 文件管理的功能

- 文件管理的功能是：
- 1. 分配与管理外部存储器，用户以文件形式存放信息，“按名存取”，文件的机内码与磁盘、光盘等外存的地址建立起相对应的表格联系。
 - 2. 提供合适的存储方法，例如，键盘命令以及程序中使用系统调用控制，包括文件的创建(create)、打开(open)、关闭(close)、读写(read/write)、删除(delete, erase)和重命名或改名(rename)等。
 - 3. 文件的共享与保护，解决文件命名中的冲突和存取权限的控制。

3.1.3 文件的组织结构

1. 文件 (file)

- (1) 文件是软件机构，软件资源的管理方式；
- (2) 具有符号名的一组相关元素的有序序列，是一段程序或数据的集合；
- (3) 一组赋名的相关联字符流的集合，或者是相关记录。而记录是有意义的信息集合。

2. 文件元素 (file Element)

- (1) 物理上：由位 (bit)、字节 (byte)、块 (block)、页 (page)、盘 (disk) 等组成；
- (2) 逻辑上：有字 (word)、字段 (segment)、记录 (record)、虚拟块 (virtual block)、文件 (file)、文件目录 (file directory)、文件夹 (folder) 和文件卷 (volume) 等。

3. 文件系统 (file system)

包含文件管理程序和所管理的全部文件（文件与目录的集合）。

- (1) 操作系统文件的特点
 - (a) 本身做成文件，每次系统启动时从外存引导其核心部分（内核）进入内存某固定位置，常驻内存；
 - (b) 仅涉及最简单的逻辑组织；
 - (c) 与数据结构 (data structure, DS)、数据库 (data base DB) 中的文件内容对比见表 3-2。

OS	无结构	无含义	基本上是字符流，提供可写的“空白笔记本
DS	有结构	无含义	提供带格的“空白本”（栈、队、表、树、图）
DB	有结构	有含义	提供带纵横项目内容格式的“空白账本“

表 3-2 文件内容对比表

- (2) UNIX 或 Linux 操作系统中文件系统的特点
 - (a) 分层次、倒树形（根、茎、叶）文件系统；
 - (b) 基本结构由一个基本文件卷（或称主文件卷）和若干子文件组成。

主文件卷是整个文件系统的根，而子文件是主文件卷的一个目录。子文件卷是通过 mount 安装到主文件卷上去的。每个用户的子文件卷可以是“树”的一个分支，独立装卸；

- (c) 文件权限保护性能好。

UNIX 将文件使用者分为三个组，分别为文件主 (user)、同组 (group) 和其他 (other)，每个组的用户均被赋予读 (read)、写 (write) 和执行 (execute)
 - (d) 设备是具有特殊文件名的文件
- 其优点是系统可以对设备和文件实施统一管理，以致大大简化设备管理程序和文件系统的接口设计。

(3) DOS 文件管理功能简介

- (a) 文件控制块功能 (FCB function)

它与 CP/M 操作系统兼容，它是在应用程序中定义的文件控制块结构，与 DOS 的文件管理功能建立传递信息的接口。利用这个接口，应用程序既能建立、打开、关闭和删除文件，也能存取该文件中任意位置和任意长度的记录。缺点是它只能存取当前驱动器上当前目录下的文件。

(b) 文件句柄功能 (file handle functions)

它以一个以零结尾的 ASCII 码字符串传递给 DOS，该码串指出了待建立或打开的文件的路径名、主文件名和扩展名。如果文件建立或打开成功，则 DOS 返回给应用程序一个文件句柄 (file handle)，它是一个十六位的二进制代码，代表一个已被打开文件的同道号。它支持树型文件结构，允许应用程序对在任何驱动器上的任意子目录下的文件进行建立、打开、读写、关闭和删除功能。

(c) DOS 文件管理功能包含文件控制块功能、文件句柄功能、目录操作功能、网络文件管理功能、盘驱动器控制功能和程序结束与取置功能。

3. 2 文件分类

1. 按文件性质与用途分类

(1) 系统文件：由系统软件构成的文件。只允许用户通过系统调用或系统提供的专用命令来执行它们，不允许对其进行读写和修改。这些文件主要有操作系统核心和各种系统应用程序或实用工具程序和数据组成。例如：msdos.sys、io.sys、及 UNIX 系统下的核心文件 /unix。

(2) 库文件：这类文件允许用户对其进行读取和执行，但不允许对其进行修改，主要由各种标准子程序库组成。如：C 语言的 *.LIB、UNIX 系统下的 /lib、/usr/lib 目录下的文件。

(3) 用户文件：这类文件是用户通过操作系统保存的用户文件，由文件的所有者或所有者授权的用户才能使用，用户将这些文件委托和操作系统保管。主要由用户的源程序、可执行目标程序、用户数据库组成。如：*.c、*.dbf、*.o 等。

2. 按操作保护分类

(1) 只读文件：只允许文件主及被核准的用户去读文件，而不允许写文件。如 -r-----。

(2) 可读写文件：允许文件主及被核准的用户去读和写文件。如：-rw-----。

(3) 可执行文件：允许文件主及被核准的用户去调用执行文件而不允许读和写文件。如：-----x-----。

(4) 各个操作系统的保护方法和级别有所不同。

DOS 操作系统的文件保护有三种：系统 (system, S)、隐藏 (hide, H)、可写 (write, W)。

UNIX 操作系统的文件保护有九种，即分三组，分别为文件主 (user)、同组 (group)、其他 (other)。每组均有 r、w、x 的权限控制。

3. 按使用情况分类

(1) 临时文件 (temporayr file)：用于系统在工作过程中产生的中间文件，一般有暂存的目录，如：\temp、/tmp、/temporary file，正常情况下，工作完毕后回自动删除。异常中断时可能会残留一些临时文件。

(2) 永久文件：指受系统管理的各种系统文件和用户文件，经过安

装、编辑、编译生成的文件，存放在软盘、硬盘、光盘等外部设备上。

(3) 档案文件：系统或一些使用工具软件包在工作过程中记录在案的文档资料文件，以便查阅历史档案。如：*.hst, *.log, *.CHK 等。

4. 按用户观点分类 (UNIX 或 Linux 操作系统)

(1) 普通文件 (常规文件)：是系统中最一般组织格式的文件，用 `ls -l` 命令列文件清单时，首项特征为：“-”，如：-rwx-rx--xx。普通文件包含系统文件、用户文件和库函数文件、实用程序文件等。

(2) 目录文件：是由文件的目录信息构成的特殊文件，操作系统将目录也称为文件，便于统一管理。这类文件包含两类信息，文件名和节点码 (i-node)，用 `ls -l` 命令列文件清单时，首项特征为“d”，如：drwx-rx-rx。

(3) 特殊文件 (设备驱动程序)：在 UNIX 或 Linux 中所有的 I/O 设备都被看成特殊文件，通过链接方式，它与设备驱动程序紧密相连。用 `ls -l` 命令列文件清单时，首项特征为“c” (字符型) 或“b” (块型)。

5. 按存取的物理结构分类

(1) 顺序 (连续) 文件：文件中的记录，顺序地存储到连续的物理块中，顺序文件中所记录的次序，与它们存储在物理介质上存放的顺序是一致的。如：存放在磁带上的文件。

(2) 链接文件：文件中的记录可存放在并不相邻的各个物理块中，通过物理块中的链接指针组成一个链表管理，形成一个完整的文件。又称为直接存取文件或指针串联文件。

(3) 索引文件：文件的记录可存储在并不相邻的各个物理块中，记录和物理块之间通过索引表项按关键字存取文件，通过物理块中的索引表的管理，形成一个完整的文件。

6. 按文件的逻辑存储结构分类

(1) 有结构文件：由若干个记录所构成的文件，又称为记录式文件。根据记录的长度特点又可分为定长记录文件和可变长记录文件。如：目前常用的数据库文件大都是定长记录文件。

(2) 无结构文件：这是直接由字符序列所构成的文件，又称为流式文件。一般来说，操作系统就是这种文件结构。可以把流式文件看成是记录文件的特例，即其中每个记录只含有一个字符。

7. 按文件的数据形式分类

(1) 源文件：是指源程序和数据构成的文件。一般由 ASCII 码、EBCD 码或汉字编码组成。

(2) 目标文件：由源程序经过相应的计算机编译程序生成，但尚未经过链接程序链接的目标代码所形成的文件，它属于二进制文件，内部地址为相对地址。通常目标文件为 *.OBJ (DOS)、*.o (UNIX)。

(3) 执行文件：目标文件经过与计算机系统提供的库函数及相关的子程序链接后形成的文件。它是二进制文件，可在操作系统的支持下运行。

8. 多媒体文件格式简介

(1) 多媒体的主要技术手段

- (a) A/D、D/A 转换技术
 - (b) 文件压缩技术及压缩算法
- 主要有 MPEG (motion photographic expert group) 的 MPEG1、MPEG2 及 MPEG3
- (2) 声音文件
- WAV 文件: Windows 提供的声音文件格式;
 - MIDI 文件: MIDI-musical instrument data interface
 - au 文件: UNIX 支持的声音文件
 - mp3 文件: 采用 MPEG3 算法
- (3) 图像文件
- BMP 文件: 位图文件
- 其它有 JPG 文件、IMG 文件、MPEG 文件。
- (4) 超文本文件
- .htm, *.html, HTML-hypertext markup language 超文本链接标记语言是 Internet 的标准文本格式。使用 HTML 格式还可以在电子邮件中添加图形和指向 Web 网站的链接 (超级链接)。
- *.PDF (portable document format): 是 Adobe 公司开发的文件格式。目前世界上不少文档说明书采用这种格式。
- (5) 其它文件

3. 3 文件的物理结构和逻辑结构

3. 3. 1 文件的物理结构

文件的物理结构又称为文件的存储结构,它是指文件在外存上的存储组织形式,是与存储介质的存储性能有关。常用的物理结构有

1. 连续文件结构

它是最简单的物理文件结构,它把一个在逻辑上连续的文件信息依次存放到物理块中。如图所示 3-a

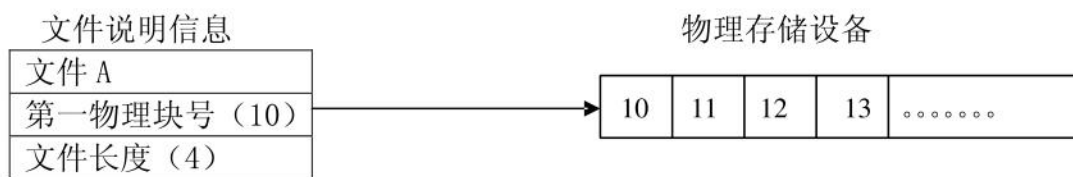


图 3-a 连续文件的物理结构示意图

2. 串联文件结构

串联文件结构用非连续的物理块来存放文件信息。这些非连续的物理块之间没有顺序关系,其中每一个物理块设有一个指针,指向其后续物理块,从而使得存放同一文件的物理块链接成一个串联队列。如图 3-b 所示

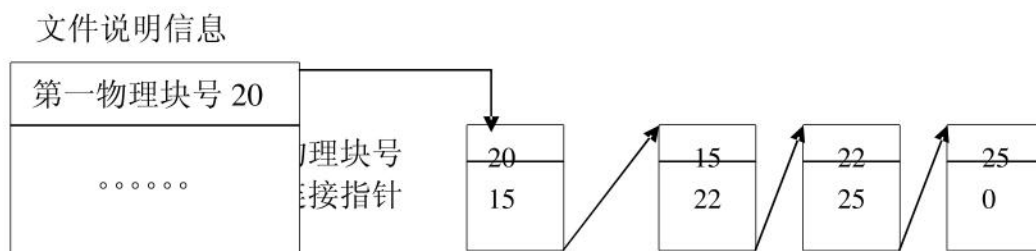


图 3-b 串联文件的物理结构

3. 索引文件结构

索引文件要求系统为每个文件建立一张索引表，表中每一栏目指出文件信息所在的逻辑块号和与之对应的物理块号。如图 3-c

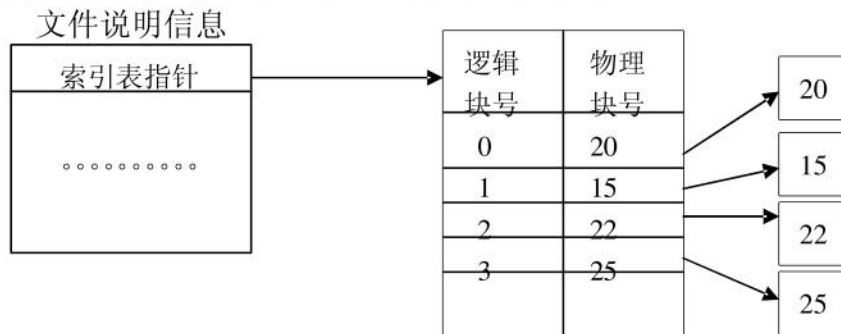


图 3-c 索引文件物理结构示意图

3. 3. 2 文件存储空间管理

由于文件存储设备是分成若干个大小相等的物理块，并以块为单位来交换信息的，因此，文件存储空间的管理实质上是一个空闲块的组织和管理问题，它包括空闲块组织，空闲块的分配和空闲块的回收等几个问题。

1. 空闲表法（空闲文件目录）

操作系统为磁盘外存上所有空闲区建立一张空闲表，每个表项对应一个空闲区，空闲表中包含序号、空闲区的第一块号、空闲块的块数等信息。它适用于连续文件结构，其分配的方式是，在系统为某个文件分配空闲块时，首先扫描空闲表项，如找到合适的空闲区项，则分配给申请者，并把该项从空闲表中去掉。如果一个空闲区项不能满足申请者的要求，则把空闲表中的另一项分配给申请者（连续文件结构除外）。如果一个空闲表项所含块数超过申请者要求，则为申请者分配了所要的物理块后，再修改该表项。

2. 空闲链表法（自由链表法）

在管理的线性表中，每一个表项对应一个空闲区或空闲块号，增加一项存放指向空闲块的指针。将磁盘上的所有空闲区（可包含若干个空闲块）拉成一条链。每个空闲区上除含有用于指示下一个空闲区的指针外，还有本空闲区块数大小的信息。

3. 成组链接法

以上两种方法由于空闲表太长而不适合大型文件系统的使用。成组链接法是两种方法相结合的一种管理方法，兼备了两种方法的优点而克服了两种方法的缺点。以下以 UNIX 操作系统为例，说明成组链接法的原理。

(1) UNIX 操作系统采用索引结构存放文件物理块的地址

在 UNIX 操作系统中，一个块定义为 512 bytes，每一个文件对应一个唯一的机内代码，称为 i 节点，在文件的 i 节点中，放有存放文件的物理块号，由对应文件的逻辑字节偏移量计算出逻辑块号后，就可搜索 i 节点的地址索引结构而得文件的物理块号。

(2) UNIX system 的 i 节点中, 定义有一个 40 字节长的字符数组。
 由于每个物理块占用 3 个字节, 因此, 此字符组最多可以放下 13 个物理块号。
 显然, 对于大于 13 个物理块 (6.5KB) 的文件, i 节点中的索引数组将无法存放所有的物理块号。

(3) UNIX 系统把文件分为小型、中型、大型和巨型四种。

(4) UNIX 系统文件的物理结构如图 3-1

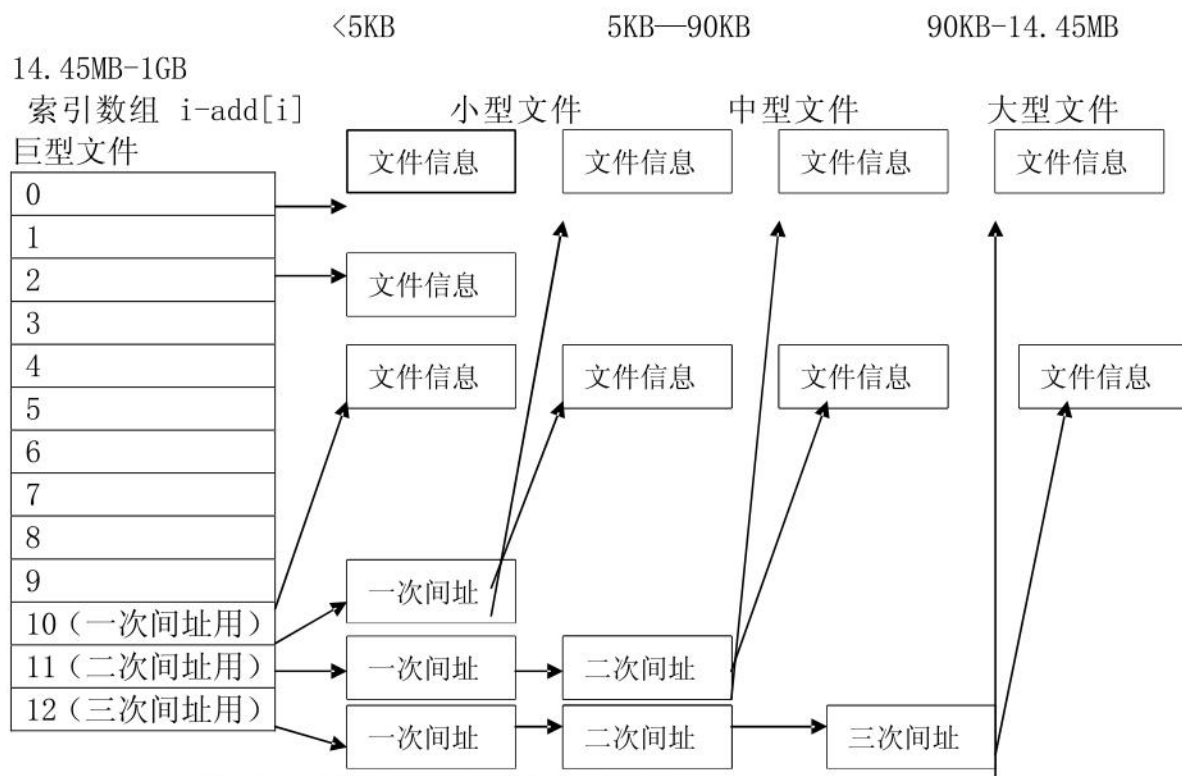


图 3-1 UNIX 成组链接文件示意图

(5) UNIX 的空闲块管理也使用类似的方法

5. 各种操作系统的分区格式

(a) DOS 的分区

DOS 将磁盘分为三个区, 分别为保留区, 控制区和文件区. 其中, 保留区为引导记录与磁盘参数表, 控制区为 FAT 和 FDT, 文件区为文件内容。

(b) UNIX 的分区

UNIX 文件系统的分区格式为 NFS(SCO OPEN SERVER 3.0)或 HTFS(SCO OPEN SERVER ENTERPRISE 5.0)

(c) Windows NT 的分区格式为 NTFS

3. 3. 3 文件的逻辑结构

1. 文件的逻辑结构 (file logical structure)

它是用户所观察到的文件组织形式，是用户可以直接处理的数据及结构，它独立于物理特性，又称为文件组织（file organization）。文件的逻辑结构可分为两大类：字符流式的无结构文件和记录式的有结构文件。

2. 文件逻辑结构设计的原则

- (1) 对于大批量的系统和用户文件，文件中又有大批有用的记录，文件的组织应有利于提高检索记录的速度和效率；
- (2) 应便于修改，便于在文件中增加、删除和修改一个或多个记录。
- (3) 尽量减少文件占用的存储空间，存放是不要占用大片连续的存储空间，以降低文件的存储开销。
- (4) *用户对文件信息进行修改时，给定的逻辑结构应能尽量减少对已存储好的文件信息的变动。

3. 文件空间管理举例

- (1) 在目录中加空白文件。
- (2) 使用索引表数据结构的自由区映象表，随机文件分别对应索引块。
- (3) 利用位视图即字位映象图。

3.4 文件目录

我们把一个文件的文件说明信息称为该文件的目录。它是文件系统中的主要数据结构之一，文件目录的管理的任务是解决存储空间的有效利用、文件快速搜索、文件命名冲突、文件共享等问题。

3. 4. 1 一级文件目录

一级文件目录的构成为一个磁盘一个目录，每一个文件在目录中有一个说明表目。优点是简单，缺点是无法防止重名或被删，安全保密性差，目前已淘汰。如表 3-3

文件名	记录长度	记录数	起始块号	其它
ABC	300	4	15	
ZWJ	400	7	29	
CMA	100	3	40	

表 3-3 一级文件目录举例

3. 4. 2 二级文件目录

二级文件目录为文件主目录（MFD）的表目按用户分，每个用户有一个用户文件目录（UFD）。优点是允许重名，提高搜索速度，缺点是不太适合大量用户和大量文件的大系统。图 3-2 是目录管理的示意图。

YANG 用户文件目录

文件名	属性	文件物理位置
ABC	。。。。	
BETA	。。。。	
ABB	。。。。	

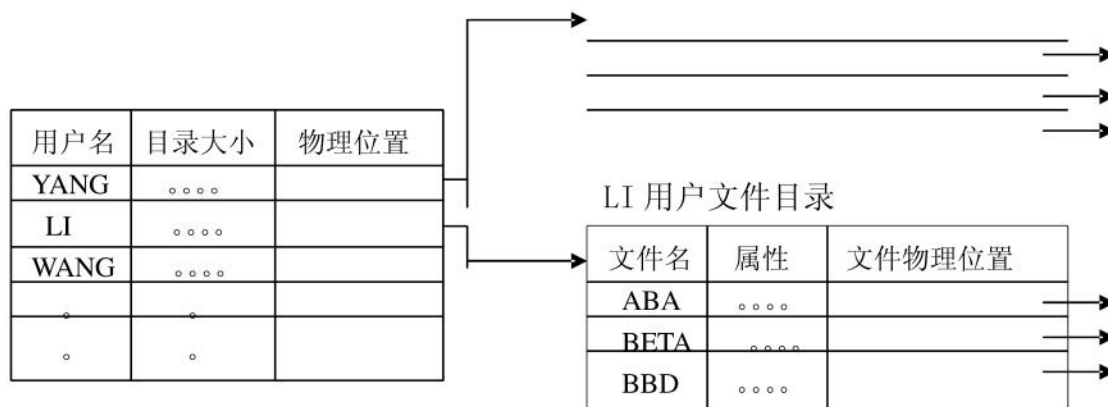


图 3-2 二级目录管理示意图

3. 4. 3 多级文件目录

多级文件目录是二级文件目录的扩充，产生于 UNIX 系统，目前已广泛应用于各种操作系统。它将目录和文件放在一起，将目录也做成文件，形成树形多层次的目录结构。

UNIX 操作系统中的文件卷是文件、层次结构目录和 i 节点（i-node）的三者组合体，它由“全路径”可确定唯一文件。硬盘可以在逻辑上划分成多个文件系统的文件卷，利用命令/etc/mount, /etc/umount 可方便地动态装卸，多级目录结构如图 3-3 所示。

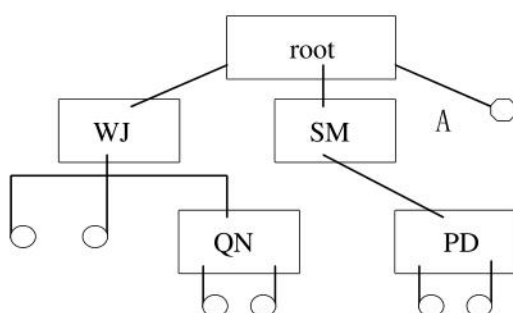


图 3-3 多级目录管理示意图

3. 4. 4 文件目录的管理

在操作系统中，一般是采用表格这种数据结构对目录进行管理，至少有打开文件表和关闭文件表，所谓打开文件是建立文件与该文件的联系，用时复制到内存。关闭文件则是断开用户与文件的联系，文件内容有变动时对外存存储。

在 UNIX 操作系统中，目录管理采用由基本文件目录表（BFD）和符号文件目录表（SDF）形成层次结构的文件目录管理，基本文件目录表又称为 i 节点（i-node），UNIX 操作系统还用活动文件表来加快文件的存取效率。所谓活动文件表即是在内存中开辟了一个活动 i 节点表区，在打开文件对文件进行读写的同时，磁盘 i 节点表区中 i 节点的信息就复制到内存活动 i 节点项中。如果需要访问文件，只需到活动 i 节点表区中去寻找文件信息。因此，盘上索引 i 节点（BFD）是文件系统操作和控制文件的依据。

3.5 文件存取控制

文件存取控制的目的是解决文件保护、保密和共享。

3.5.1 存取控制矩阵

用二维表格来进行文件的存取控制，一维列出系统全部用户，另一维列出系统全部文件，如果矩阵元素是“1”，表示对应用户允许访问，反之，“0”为不允许，如表 3-4 所示。

文件 用户	A	B	C	D
甲	1	1	1	0
乙	0	1	0	0
丙	0	0	1	1

表 3-4 存取控制矩阵

3.5.2 用户权限表

用户权限表是以用户为单位或以文件为单位，对每一类用户规定权限，在 UNIX 操作系统中，用户可分为三类，分别为文件主（owner）、同组人（group）和其他人（other），每一类又可分为可读（r）、可写（w）和可执行（x），共有 9 种控制状态。如表 3-5

文件 用户	A	B	C	D
甲	rx	r	r	x
乙	r	rx	r	x
丙	rw	rw	r	x

表 3-5 用户权限表例

3.5.3 使用口令

方法是对文件系统和网络用户分组设置用户登录名（username）和口令（password）。优点是占用存储空间小、方便，缺点是保护能力弱。

3.5.4 使用密码

存储时用“密钥”（密码钥匙）对文件进行编码，读取时译码。例如，在 UNIX 操作系统中由命令 crypt 可以对文件进行加密和解密。

3.5.5 UNIX/Linux 操作系统的安全性

1. UNIX 系统的安全机制

(1) 口令（password）

口令是 UNIX 系统的最基本的保护机制。要成功地在 UNIX 操作系统上注册，必须正确的用户名（username）和口令（password），否则系统将拒绝用户进入系统。UNIX 用一个加密的口令文件/etc/passwd 保存所有用户口令。

(2) 文件权限控制

(3) 用 crypt 命令对文件进行加密

命令格式

- (a) 加密 `$crypt [口令]<文件 1> 文件 2`
- (b) 解密 `$crypt [口令]<文件 2> 文件 1`

2. UNIX 的不安全因素

- (1) UNIX 系统允许用户不设置口令。
它会导致非法用户盗用其用户名进入系统，从而破坏该用户的文件。
- (2) 口令猜测程序会使用户的口令被破译。
- (3) 文件权限的设置不当可能增加文件的不安全因素。
- (4) “特洛伊木马”程序可能会导致用户口令和用户权限的被盗。

3. 系统管理员及用户的职责

- (1) 系统管理员
- (2) 用户

3. 5. 6 计算机病毒简介

1. 计算机病毒的概念

计算机病毒是人为编制的一种计算机程序，其活动方式和生物学中的病毒活动方式相类似。

2. 计算机病毒的特征

- (1) 破坏性；
- (2) 寄生性；
- (3) 传染性；
- (4) 隐蔽性；
- (5) 潜伏性。

3. 计算机病毒的类型

- (1) 引导型病毒
- (2) 文件型病毒
- (3) 混合型病毒

4. 计算机病毒的危害

- (1) 破坏文件分区表
- (2) 删除磁盘文件
- (3) 修改或破坏文件中的数据
- (4) 占用大量内存，降低系统的运行速度
- (5) 对磁盘的特定扇区或磁道以至整个磁盘进行格式化；
- (6) 减少磁盘的可用空间；
- (7) 改变计算机的正常工作进程；
- (8) 产生人为的文件，占用磁盘空间；
- (9) 扰乱屏幕显示；

5. 计算机病毒的防治

- (1) 计算机病毒的预防
计算机病毒的预防主要采取技术手段和管理手段相结合的方法。
- (2) 计算机病毒的消除

3. 6 文件系统的数据结构和表示

3. 6. 1 UNIX/Linux 文件系统

UNIX 操作系统的特点

1. 文件的目录组织是一个树形结构, 从根节点到叶子称为文件的路径名, 文件可以由其路径名唯一确定。
2. 文件本身是无结构的字符流。
3. 文件系统把外部设备的特殊文件和普通文件以及目录文件都统一在文件这一概念之下。

3. 6. 2 磁盘文件系统结构

1. UNIX 操作系统文件系统的结构

UNIX 操作系统中, 磁盘在逻辑上划分成多个文件系统的文件卷, 每个文件卷都可存放一个具有独立的目录结构的文件系统, 但不能将一个文件卷分散放在多个存储介质上。文件卷按物理块号排列的结构如图 3-4。

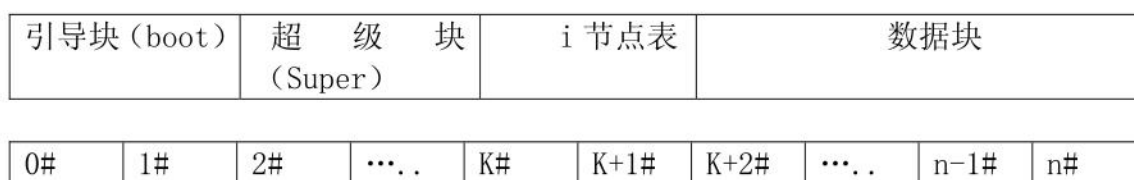


图 3-4 文件卷组织结构图

(1) 0 号块为系统引导块或空闲, 放在文件系统的开始。它包含的引导码用来读入这台机器的引导程序, 初始化操作系统。

(2) 1 号块又称为超级块, 它存放整个文件卷的资源管理信息, 如文件卷的大小、有多少个文件、在哪里能找到空闲块等。

(3) 从第 2 块起直到 K+1 块存放索引节点, 称为磁盘 i 节点, 其中存放文件描述信息。

(4) 第 K+2 到第 N 块称为数据块, 它包含文件数据和管理数据。在文件系统中一个分配的数据块只能属于唯一的一个文件。

2. 超级块的存储结构

```
struct filsys
{
    unsigned short s-isize; /*记录该文件系统磁盘 i 节点的块数*/
    long s-fsize;          /*整个文件系统的块数*/
    short s-nfree;         /*空闲磁盘块栈指针*/
    daddr_t s-free[NICFREE]; /*空闲磁盘块栈*/
    short s-ninode;        /*空闲 i 节点栈指针*/
    ino_t s-inode[NICINOD]; /*空闲磁盘 i 栈指针*/
    char s-flock;          /*空闲磁盘块栈的上锁标志*/
}
```

```

char s-ilock           /*空闲磁盘 i 管理的上锁标志*/
char s-fmod;           /*专用块修改标志*/
char s-ronly;          /* 文件卷只读标志*/
time_t s-time;         /*专用块最近一次修改时间*/
short s-dinfo;         /*设备信息*/
daddr_t s-tfree;       /*该文件卷中所有空闲块总数*/
ino_t s-tinode;        /*该文件卷中空闲磁盘 i 节点总数*/
char s-fname[6];       /*文件系统名字*/
char s-fpck[6];        /*文件系统经压缩的名字*/
long s-magic;          /*新文件系统的幻数*/
long s-type;           /*新文件系统的类型*/
}

```

3. 6. 3 文件系统的数据结构

1. 磁盘 i 节点的数据结构

在 UNIX 操作系统中，文件的有关管理信息与目录项是分开的，每个文件控制块是一个称为 i 节点的数据结构。在文件存储设备上设立了一个专门的 i 节点区，该区中 i 节点顺序编号。1 号节点保留不用，2 号（ROOTINO）i 节点用于存放文件系统的根目录。磁盘 i 节点的数据结构如下：

```

struct dinode
{
    unsigned short di-mode;    /*文件模式*/
    short di-nlink;           /*文件连接数*/
    unsigned short di-uid;     /*文件所有者用户标识符*/
    unsigned short di-gid;     /*同组用户标识符*/
    long di-size;             /*文件容量*/
    char di-addr[40];         /*磁盘块号索引表*/
    long di-mtime;            /*文件最近修改时间*/
    long di-ctime;            /*文件最近存取时间*/
    long di-ctime;            /*文件建立时间*/
}

```

2. 内存 i 节点的数据结构

磁盘 i 节点结构存储在外存上不能直接访问。为了提高工作效率，UNIX 操作系统在内存中设置了内存 i 节点结构。内存 i 节点的数据结构如下：

```

struct mode
{
    struct inode *i-forw;    /*hash 链前向指针*/
    struct inode *i-back;    /*hash 链后向指针*/
}

```

```

char flag;           /*文件活动标志*/
cnt-ti-count;       /*访问记数*/
dev-ti-dev;         /*i 节点所在设备名*/
ino-ti-number;      /*i 节点号*/
ushorti-mode;       /*文件模式*/
short i-nlink;      /*文件连接数*/
ushort i-uid;        /*文件所有者用户标识*/
ushort i-gid;        /*同组用户标识符*/
off-ti-size;        /*文件大小*/
struct{
    union{
        daddr-ti-a[NADDR]; /*地址索引表*/
        short i-f[NADDR];  /*用于先进先出文件，即 pipe 文件*/
    }i-p;
    daddr-ti-l;             /*上次读的逻辑块号*/
}I-blks;
}

```

1.7 文件系统的系统调用

UNIX 操作系统提供的系统调用有 56 个，其中涉及文件系统的系统调用 20 个。进入系统调用的工作是在陷阱指令 trap 中完成的，文件的基本操作有：

1. 文件的创建 (create)：即构造一个新文件，可通过文件描述字或文件的路径名指明文件的性质，参数表包含文件路径名指针和新文件的名字。
2. 文件的打开 (open)：即打开一个文件，文件从外存调入内存。
3. 文件的读 (read)：查看已打开的一个文件内容或读出传送。
4. 文件的写 (write)：对一个已打开的文件，写入新的内容或修改文件内容。
5. 文件的关闭 (close)：将已打开的文件从内存送回外存，如果文件内容没有变动，操作系统不再对外存重写，仅仅是腾出内存空间。
6. 文件的删除 (delete)：永久性地删除一个文件，包括解除文件的联结，即联结数为零时，则删除此文件。

3.8 重点小结

1. 文件系统为用户提供按名存取的功能，以使用户能透明地访问存储文件。文件系统需要对文件存储设备进行合理的组织、分配和管理；对文件进行检索、保护、保密和提供共享手段。
2. 文件的逻辑结构可分为字符流式无结构的连续文件和记录式的有结构文件两大类。文件的物理结构有连续文件结构、串联文件结构、索引文件结构，它们和存储设备的类型密切相关。
3. 文件名或记录名与物理地址之间的转换是通过文件目录来实现的。有单级目录、二级目录和多级目录等几种目录结构，现普遍使用的是多级目录结构。
4. 对文件的存取控制是和文件共享、保护和保密紧密相关的。存取控制可采用存取控制矩阵、存取控制表、口令和密码的方法进行存取验证，以确

定用户权限。

第 4 章 存储管理

存储器是计算机系统的重要资源之一。因为任何程序和数据以及各种控制用的数据结构都必须占用一定的存储空间，因此，存储管理直接影响系统的性能。

存储器由内存和外存组成。内存由顺序编址的块组成，每块包含相应的物理单元。CPU 要通过启动相应的 I/O 设备才能使外存和内存交换信息。本章主要讨论内存管理问题。

4. 1 存储管理的任务和功能

4. 1. 1 存储管理的任务

- (1) 方便用户，使用户减少甚至摆脱对存储器使用的管理；
- (2) 提高内存资源的利用率，关键是实现资源共享。

4. 1. 2 存储管理的功能

- (1) 内存区域的分配和管理：通过建表、查表、改表和回收登录内存使用情况。系统或用户申请内存时按选定的分配算法确定分区等。
- (2) 内存的扩充技术：使用虚拟存储或自动覆盖技术提供比实际内存更大的空间；
- (3) 内存的保护技术：各道作业、任务或进程在自己所属区域中运行，不破坏别的作业或不被“别人”破坏，更不要破坏整个系统工作。

4. 1. 3 内存类型及寻址

内存产品类型简表见表 4-1

线 数	型 号	简 称	其 他
30 线	SIMM (Single IN-Line Memory Modules)	DRAM	最大 16MB
72 线	SIMM	EDO DRAM	速度 80/70/60ns 等
168 线	DIMM(Dual IN-Line Memory Modules)	EDO DRAM SDRAM	8/16/32/128MB 等

表 4-1 内存产品类型简表

1. 常规（基本）内存

- (1) DOS 作为单用户的操作系统，它是以实模式为运行环境的（8088/8086 的地址总线为 20 位，CPU 寻址空间为 1M），DOS 管理的常规（基本）内存最多为 640KB，不加“处理”的 DOS 版本根本感觉不到物理内存的扩充。
- (2) DOS 扩展的内存管理（EMM）功能
 - (a) EMM—expand memory management 是 Lotus、Intel 和 Microsoft 公司制定的扩展内存管理规范，它是为了发挥 80286 能够寻址 16M 空间而制定的。
 - (b) EMM 管理程序以 INT 67H 的方式向应用程序提供 01—0FH 共 15 个子功能。应用程序要使用扩展内存空间，如同使用系统功能调用（INT 21H）那样，只需给出相应的入口参数即可。
 - (c) 调用 INT 67H 的各个子功能之前，必须确保 EMM 扩展内存管理程序已加载

至内存，EMM 程序是以可安装的字符设备驱动程序的形式，通过配置文件 CONFIG.SYS 中的 DEVICE 命令项内容在 DOS 初始化期间被加载的。

(d) EXEC 是 DOS 的一个重要的子功能。它以 INT 21H 的 4BH 功能号形式出现，供系统程序或应用程序（称为父程序）加载另一个程序（称为子程序）到内存并执行，子程序结束退出又返回到父程序。

(f) EXEC 是从磁盘加载一个子程序到内存然后执行，CALL 是调用已在内存的子程序。

2. 扩展内存 (expanded memory, EMS)

DOS 将 640KB 至 1024KB 的 384KB 称为高端存储区 (UMB)，一般用它作为存放硬件适配器驱动程序，如 VGA 显示器驱动程序。DOS5.0 以上的版本对大于 1088KB 的内存管理，采用分页映射，象卡片式的存储体开关技术仿真扩充内存进行管理，称为扩展内存(EMS)。它由 CONFIG.SYS 中增加语句 DEVICE=EMM386.SYS 后管理生效。这一语句应放在 HIMEM.SYS 后才生效。

3. 扩充内存 (extended memory, XMS)

DOS 将内存 1024KB 至 1088 KB 区称为扩充内存管理 (XMS) 程序区，由在 CONFIG.SYS 文件中增加语句 DEVICE=HIMEM.SYS 后管理生效。

4. 辅助存储器 (auxiliary memory)

为了弥补主存储器的容量不足而使用的一种存储器。在计算机的控制下，它和主存储器可以成批地自动交换数据和指令。

5. 快存 (cache memory)

快存是一种高速缓冲存储器，在内存和 CPU 之间工作的快速小容量存储器。它可以改善内存的有效传输率，从而提高计算机的运行速度。目前，一般的方法是，计算机磁盘子系统的常用数据缓存在主存中，主存中的常用代码和数据缓存在二级高速缓存中，最常用的代码和数据存储在 CPU 的一级高速缓存器中。

6. 逻辑（相对）地址与物理（绝对）地址

逻辑地址 (logical address、relative address)

程序中按逻辑顺序编排的代码及数据的地址称为逻辑地址。

物理地址 (physical address、absolute address)

程序中按代码及数据在内存中实际存储位置的地址成为物理地址。

7. 重定位 (relocation)

将逻辑地址转化为物理地址的过程称为重定位。一般由操作系统的链接过程完成。分为静态和动态两种。

静态链接是在链接装入时一次集中完成，动态是在指令执行中先访问内存后再重定位，一般由硬件完成地址转换。重定位原理图见图 4-1。

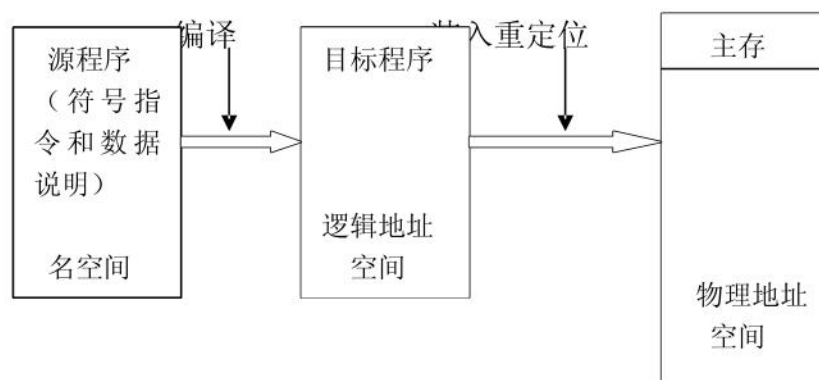


图 4-1 重定位原理图

8. 虚拟存储 (virtual memory)

(1) 内存和外存的特点

内存和外存的特点见表 4-2

内存	速度快	容量小	价格高	受地址总线限制
外存	速度慢	容量大	价格低	间址

表 4-2 内存和外存的比较

(2) 虚拟存储

编译链接程序把用户源程序编译后链接到一个以 0 地址为始地址的线性或多维虚地址空间。每个指令或数据单元都在这个虚地址空间中有确定的地址，我们把这个地址称为虚拟地址 (virtual address)。我们把由进程中的目标代码、数据等的虚拟地址组成的虚拟空间称为虚拟存储器 (virtual memory store)。虚存是由操作系统调度，采用内外存的交换技术，各道程序在必需时调入内存，不用时调出内存，这样好象内存容量不受限制。

(3) 虚拟存储的制约因素

(a) 虚存容量不是无限的，极端情况受计算机内、外存总和的限制；

(b) 虚存容量还受计算机总线长度和地址结构的限制；

(c) 速度和容量的“时空”矛盾，虚存容量的“扩大”是以牺牲 CPU 工作时间以及内、外存的交换时间为代价的。

9. Windows 的三种工作模式

(1) 实模式

实模式的特点是使用 16 位寄存器、段寄存器中存放 16 位段地址，段地址是参与寻址的，段地址左移 4 位，加上偏移地址，得到 20 位的物理地址，使得 CPU 可定位的最大物理地址为 $2^{20}=1024\text{KB}=1\text{MB}$ 。

(2) 保护模式

保护模式的特点是使用 32 为寄存器、段寄存器中存放 16 为段选择器，它的值不参与寻址，只是一个指向段描述表的索引，至少有三张段描述表：全局、局部和中断描述表。理论上讲在保护模式下，CPU 可寻址的物理地址有 $2^{32}=4096\text{MB}=4\text{G}$ 。

(3) V86 模式

V86 模式是一个全新的虚拟内存管理系统。这个 386 以上增强模式的虚内存申请系统是一个独立运行的系统，它位于段方式之上，一旦物理内存被添满，内存页 (4KB) 就会在磁盘中倒来倒去，最终的结果是使线性内存寻址空间可以达 64MB

10. 内存模式

四种标准的内存模式见表 4-3

	16 位数据寻址	32 位数据寻址
16 位代码寻址	小模式	紧凑模式

32 位代码寻址	中模式	大模式
----------	-----	-----

表 4-3 四种标准的内存模式

4. 2 分区分配存储管理

分区存储管理技术是满足多道程序设计的一种最简单的存储管理方法，其基本原理是给每一个内存中的进程划分一块适当大小的存储区，以连续存储各进程的程序和数据，使个程序得以并发执行。按分区的时机，分区管理可以分为固定分区和动态分区两种方法。

4. 2. 1 固定分区分配

1. 固定分区分配（fixed-size partition）是在处理作业前，内存事先划分为若干个大小不等或相等的区域，一旦划分好则固定不变，每个作业占一个分区，作业是连续存放的。分区的划分可以由操作系统或系统管理员决定。

2. 系统对内存的管理和控制通过数据结构—分区说明表进行，分区说明表说明各分区号、分区大小、起始地址和是否是空闲区（分区状态）。内存的分配释放、存储保护以及地址变换都通过分区说明表进行。分区说明表的结构如图 4-2

分区号	大小	始址	状态
1	9KB	20KB	已分配
2	25KB	29KB	可用
3	40KB	54KB	可用
4	162KB	94KB	可用

(a) 分区说明表

操作系统	进程 A (9K)	进程 B (25K)	进程 C (40K)	进程 D(94K)
0	20K	29K	54K	94K
256K				

(b) 内存空间

图 4-2 固定分区分配例图

3. 固定分区方法的优缺点

固定分配的优点是分配回收方便，适用于用户不多的小型系统；缺点是内存使用不充分，每一分区剩余部分无法利用。

4. 2. 2 可变式动态分区分配

1. 动态分区的原理

动态分区法在作业执行前并不建立分区，而是在处理作业的过程中按需要建立分区，而且其大小可随作业或进程对内存的要求而改变。这就改变了固定分区中小作业占据大分区的浪费现象，从而提高了系统的利用率。

2. 动态分区的数据结构

动态分区采用三张表对内存管理，分别为已分配区域说明表、未分配区域说明表（可用表）和资源请求表。相互的关系如图 4-3。

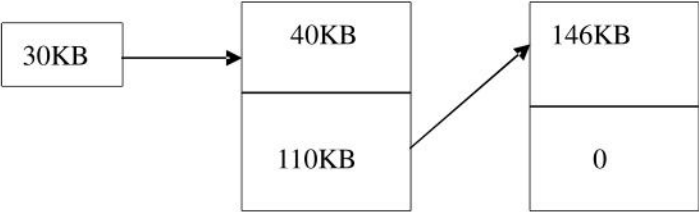
分区号	大小	始址	状态
-----	----	----	----

1	10KB	20KB	已使用
2	---	---	
3	40KB	70KB	已使用
4	---	---	

(a) 已分配区域说明表

分区号	大小	始址	状态
1	---	---	---
2	40KB	30KB	可用
3	---	---	
4	146KB	110KB	可用

(b) 未分配区说明表



(c) 自由链



作业（进程）号	请求长度
P1	13K
P2	20K

(e) 资源请求表

图 4-3 动态分区分配例图

3. 动态分区的分配与回收

(1) 分配

动态分区法在分配前，除操作系统本身占用外，只有一个空白区。分配时，按一定的算法从空白表区中找，看是否有满足作业的可用分区，如果空白区存在则分配，分配后修改两张表的内容，如果找不到满足要求的空闲区则系统报错。

(2) 分配算法

(a) 首次适应法 (first-fit)

要求把内存中的可用分区单独组成可用分区表或可用分区自由链，按起始地址递增的次序排列。查找的方法是每次按递增的次序向后找，一旦找到大于或等于所要求内存长度的分区，则结束查找，从找到的分区中划分所要求的内存长度分配给用户，把剩余的部分进行合并（如果有相邻的空白区存在的话），并修改可用区中的相应表项。

(b) 循环适应法 (circulation-fit)

系统记住上一次分配区地址，每重新分配一次时，都在当前之后寻找，其目的是回收空白区。即内存所有的线性空间可能轮流使用到。分配的时间会快一些，“碎片”也可能会小一些。

(c) 最佳适应法 (best-fit)

最佳适应法要求按空白区的大小，从小到大次序组成空白区表或自由链。寻找的方法是找到第一个满足要求的空白区时停止查找，如果该空白区大于请求表中的请求长度，则将剩余空白区留在可用表中（如果相邻有空白区，则与之合并），然后修改相关表的表项。

(d) 最坏适应法 (worset-fit)

最坏适应法要求按空白区大小，从大到小递减顺序组成空白区可用表或自由链。寻找的方法是当用户作业或进程申请一个空白区时，选择能满足要求的最大空白区分配，先检查空白区可用表或自由链的第一个空闲区的大小是否大于或等于所要求的内存长度，若满足，则分配相应的存储空间给用户，然后修改和调整空闲区可用表或自由链，否则分配失败。

(3) 动态分区时的回收和拼接

当用户作业或进程执行结束时，存储管理程序要回收已使用完毕的空闲区，并将其插入空闲区可用表或自由链。如果不对空闲区进行拼接，则由于每个作业或进程所要求的内存长度不一样而出现大量分散、较小的空闲区。这就造成大量的内存浪费。解决这个问题的办法之一就是在空闲区回收时进行空闲区拼接，以把不连续的零散空闲区集中起来。

4. 2. 3 可重定位分区分配

可重定位分区分配 (re-locatable partition) 即一旦出现分散的空白区，就将空白区集中，把已分配的分区移动紧靠一端，空白区集中在另一端。有硬件和软件两种方法。其目的是解决“碎片问题”，但以牺牲系统的效率为代价。

4. 2. 4 多重分区分配

多重分区分配是以程序分段为基础来实现的。它将程序分成若干段，如主程序、子程序和数据段等，然后以段为单位进行分区分配。这些段在逻辑上是连续的，但在存放在内存分区中去时却无需连续，只要有足够的保护措施就可以了。其优点是便于多个作业共享内存，缺点是存储分散，管理复杂。

4. 2. 5 内存扩充技术

在分区分配存储管理中常用的两种内存扩充技术有覆盖与交换技术。覆盖技术主要用于早期的操作系统中。

1. 交换 (swap)

交换技术是先在硬盘上开辟一个交换区，将内存某部分的程序或数据写入外存交换区，再从外存交换区调入指定的程序或数据到内存中来，并让其执行的一种内存扩充技术。一般包括换入和换出两个过程。

2. 覆盖 (overlay)

覆盖由操作系统或编译程序提供用户覆盖机构。覆盖机构将整个作业分为常驻和覆盖两部分。因为一般用户程序由主程序和若干个子程序组成，而子程序是不会同时调用工作的，用户只要最大的子程序作为覆盖区告诉系统，各个子程序都有足够的空间工作，因此，其它子程序就不需要事先占用内存。

4. 2. 6 内存保护技术

内存保护技术 (protection) 要保护系统工作区和用户工作区，特别是如何防止系统区被破坏。

1. 存储保护键

系统为每个分区设一保护键，在程序状态字中也设同样的保护键字段，访问内存时检查键的配对情况，如果不对则产生保护性中断。

2. 界限寄存器

由硬件实现，起重定位寄存器的作用。采用基址寄存器（存放作业的起始地址）和限长寄存器（存放作业的长度）或采用上界寄存器（存放作业的最高地址）和下界寄存器（存放作业在内存中的最低地址）。

系统调用作业工作时查内存分配表，查到分配的分区基址，计算出绝对地址，执行一条有关地址的指令，由硬件检查是否越界，决定继续访问或产生越界中断处理。

4. 3 请求页式存储管理

分区存储管理尽管实现方式简单，但存在着严重的碎片问题使得内存的利用率不高。再者，分区管理时，由于各作业或进程对应不同的分区以及在分区内各作业或进程连续存放，进程的大小仍受分区大小或内存可用空间的限制。为此提出了页式存储管理。

页式存储管理可分为静态页式管理和动态页式管理，而动态页式管理又分为请求页式管理和预调入页式管理。本章介绍动态页式管理的请求页式管理。

4. 3. 1 实现原理

1. 划分实页

将物理内存划分成位置固定、大小相同的“块”（实页面）。其特

点是：

- (1) 分页是为了管理，物理内存没有按用户作业分区概念，分页仅仅是为了信息管理构造用，为了便于提高工作效率；
- (2) 用户不可见，物理内存也没有真正隔离，即“虚拟”的隐分页，一页中的地址必须连续；
- (3) 分页是一种物理划分而不是逻辑划分单位，因此，页的共享有困难。

2. 划分虚页

将用户逻辑地址空间也分成同样大小的页面，成为虚拟空间的虚页面，其特点是：

- (1) 用户可用地址大小受物理地址大小以及地址总线的限制；
- (2) 虚页号可大于实页号；
- (3) 从概率来看有半页浪费，因为可能遇到只有一个字节也要占一页。

3. 建立页表

建立页表，有时称为页面表或页面映射表（PMT）。每个作业一张，按虚页号进行登记，其基本的内容有特征位（表示该页是否在内存、实页号以及对应外存的地址，如表 4-4

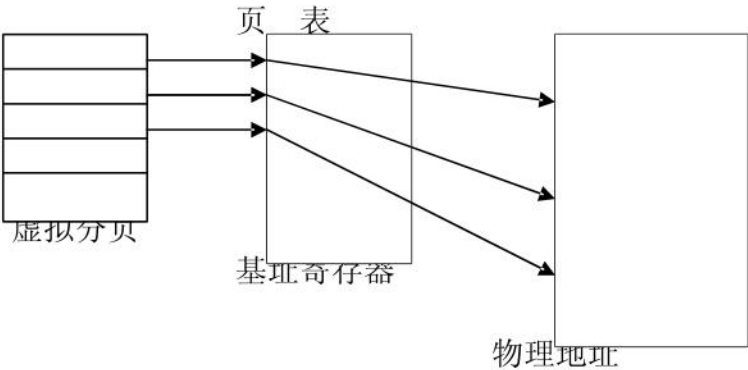


图 4-4 分页工作图

4. 地址变换

将虚页面的逻辑地址转化为实页面的物理地址，在程序执行时改变为物理地址，属于作业的动态重定位，一般由地址转换机构（硬件）完成。

4. 3. 2 页表的设计

分区存储管理技术提供三种表格进行存储管理，分别为存储分块表、作业表和页表。

1. 存储分块表

整个系统一张，记录整个内存的使用情况，如，内存目前空白块总块数以及指向第一空白块的指针。主要有位示图和空白块链两种方法。

2. 作业表

整个系统一张，每个作业占一个表项，表的内容如图 4-A 所示

作业号	请求页面数	页表始址	页表长度	状态
1	20	1024	20	已分配

2	34	1044	34	已分配
3	18	1078	18	已分配
4	21	1096	21	未分配

图 4-a 作业表示例

3. 页表

每个作业一张，其表项包括页号（虚页）、页面号（实页）中断位、改变位、淘汰位等，如图 4-B。

页号	页面号	中断位	外存始址	改变位	淘汰位

图 4-B 页表示例

4. 3. 3 请求淘汰换页算法

1. 分页存储管理要解决的问题

分页存储管理只让进程或作业的部分程序和数据驻留在内存中，因此，在执行过程中，不可避免地会出现某些虚页不在内存中的问题。请求页式管理必须解决的两个基本问题是：1. 怎样发现这些不在内存中的虚页；2. 怎样处理这种情况（采用何种方法把所缺的页调入内存，以及当内存中没有空闲页面时怎么办），1 的解决是通过在页表中设置中断位及虚页在外存中的始址来处理的；2 的解决就涉及到淘汰换页算法。

2. 缺页调度流程

缺页调度流程如图 4-6

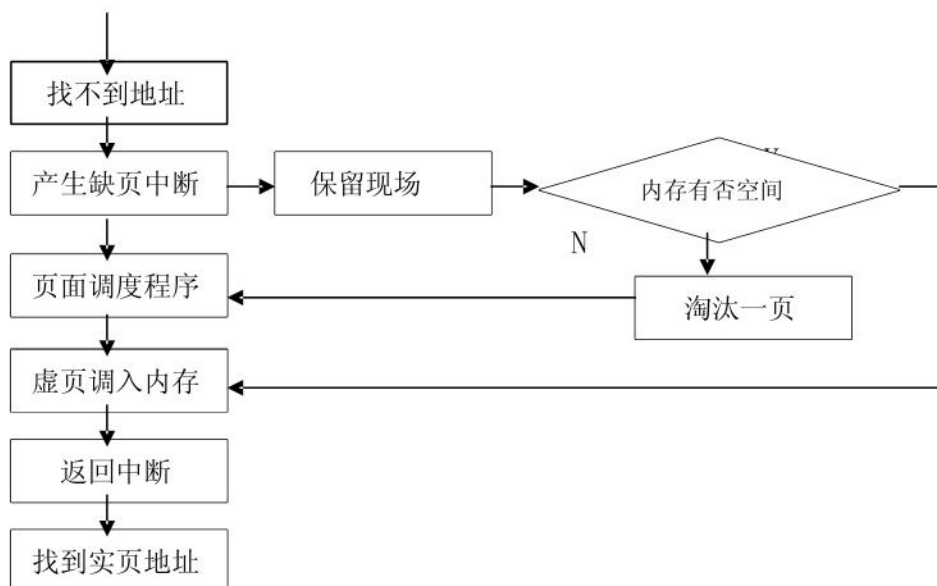


图 4-6 缺页调度流程

3. 请求淘汰换页算法

(1) 先进先出算法 (first input first output, FIFO)

先进入内存的页面先淘汰。实现是在页表中登记进入的次序，并将各个已分配的页面按分配时间顺序连接起来，组成 FIFO 队列。优点是实现简单，缺点是遇到常用的页效率低下，并可能产生 Belady 现象（所谓 Belady 现象是指分配的页面数增多，缺也次数反而增加的现象）。

(2) 循环检测法

让循环多的页面留驻内存，计算机采用记录页面住留内存期间对该页的访问时间， t 为该页上一次访问时间， T 为该页第二次访问时间，选用相对时间 $(t-T)$ 最大的淘汰。优点是适合循环多的大程序；缺点是系统开销大。

(3) 最近最少使用页面先淘汰 (least recently used, LRU)

该算法的基本思想是：当要淘汰某页时，选择离当时时间最近的一段时间内最久没有使用过的页面先淘汰。该算法的出发点是，如果某页被访问了，则可能它马上还要被访问，或者反过来说，如果某页面很长时间未被访问，则它在最近一段时间也不会被访问。LRU 的实现是一件十分困难的事情，我们一般采用它的近似算法。

(4) 最不经常使用的页面先淘汰 (least frequent used, LFU)

该算法在需要淘汰某一页时，首先淘汰到当前时间为止，被访问次数最少的那一页。这只要在页表中给每一页增设一个访问计数器即可实现。每当该页被访问时，访问计数器加 1，而发生一次缺页中断时，则淘汰计数值最小的那一页，并将所有的计数器清零。

(5) 最近没有使用的页面先淘汰 (not used recently, NUR)

它是上述算法的一种简化，利用在页表中设置一个访问位即可实现，当某页被访问时，访问位置“1”，否则访问位置“0”当需要淘汰一页时，从那些访问位为“0”的页中选一页进行淘汰。系统周期性地对所有访问位清零。

(6) 随机数淘汰页面算法 (random replacement algorithm)

在系统设计人员无法确定那些页的访问概率较低时，随机地选择某个用户的页面进行淘汰也是一种方法。

(7) 最优淘汰算法 (optimal replacement algorithm, OPT)

它是一种理想的淘汰算法，系统预测作业今后要访问的页面，淘汰页是将来不被访问的页面或者最长时间后才能被访问的页面。这种算法是无法实现的，因为它要求必须预先知道每个进程的访问串。

4.3.4 页式存储管理的优缺点

1. 优点

- (1) 虚存量大，适合多道程序运行，用户不必担心内存不够的调度操作。动态页式管理提供了内存与外存统一管理的虚存实现方式；
- (2) 内存利用率高，不常用的页面尽量不留在内存；
- (3) 不要求作业连续存放，有效地解决了内存碎片问题。

2. 缺点

- (1) 要进行页面中断，缺页中断等处理，系统开销较大；
- (2) 有可能产生“抖动”现象；
- (3) 地址变换机构复杂，一般采用硬件实现，增加了机器成本。

4.3.5 UNIX 系统的请求页管理

1. UNIX system V 采用请求页式虚存技术，并给每一也赋予存取特权。页长 512KB，内存的申请和释放均以页为单位。
2. 计算机为这种技术提供必要的硬件支持，如地址变换机构等。
3. 虚空间被划分为进程空间和系统空间，进程空间又分为程序区和控制区；系统空间又分为核心区和保留区。

4. 4 段式存储管理

1. 段式存储管理的基本思想

页式存储管理由于页的划分固定，每页的内容无完整的意义，因此页式存储管理在不同作业或进程之间共享公用子程序和数据段变得非常困难，由此提出了段式存储管理的概念。段式存储管理的基本思想是：把程序按内容或过程（函数）关系分成段，每段有自己的名字。一个用户作业或进程所包含的段对应于一个二维线性虚拟空间，也就是一个二维虚拟存储器。段式管理程序以段为单位分配内存，然后通过地址映射机构把段式虚拟存储地址转化为内存中的实际地址。和页式管理一样，段式管理也采用只把那些经常访问的段驻留内存，而把那些在将来一段时间内不被访问的段放在外存，待需要时自动调入内存的方法实现二维虚拟存储器。

2. 段式与页式的比较

见表 4-4

段 式	页 式
分段由用户设计自己划分，每段对应的程序模块，有完整的逻辑意义	分页用户看不见，由操作系统为内存管理划分
段面是信息的逻辑单位	页面是信息的物理单位
便于段的共享，执行时按需动态链接装入	页一般不能共享
段长不等，可动态装入，有利于新数据的增长	页面大小相同，位置不能动态增加
二维地址空间：段名、段中地址；段号、段内单元号	一维地址空间
管理形式上象页式，但概念不同	往往需要多次缺页中断才能把所需的信息完整地调入内存

表 4-4 段式与页式存储管理比较

4. 5 段页式存储管理

4. 5. 1 特点

- (1) 每一段分成若干页，再按页式管理，页间不要求连续；
- (2) 用分段方法分配管理作业，用分页方法分配管理内存；
- (3) 段页式管理是段式管理和页式管理结合而成，兼有段式和页式管理的优点，但系统的复杂性也随之增大，一般用在大型机上。

4. 5. 2 实现原理

- (1) 采用二维地址空间，如段号 (S)、页号 (P) 和页内单元号 (D)；
- (2) 系统建两张表格每一作业一张段表，每一段建立一张页表，段表指出该段的页表在内存中的位置；
- (3) 地址变换机构类似页式机制，只是前面增加一项段号。

4.6 重点小结

- (1) 内存管理要解决的问题有：内存的分配与释放、内存扩充、地址变换、内存的保护与共享、内外存之间数据交换的控制等问题；
- (2) 内存管理的核心问题是如何解决内存和外存的统一管理以及它们之间的数据交换问题，内存和外存的统一管理使得内存的利用率等到提高，用户程序不再受内存可用区大小的限制；
- (3) 掌握常用的内存管理方法，重点是分区存储管理和请求页式存储管理。

第5章 输入输出设备管理

本章是操作系统的第四大功能，属于对硬件的管理。主要内容有：外部设备的分类及安装、输入输出设备的分配算法、外部设备和 CPU 之间的数据传送控制方式（程序直接控制方式、中断控制方式、DMA 方式和通道方式）和设备驱动程序等。

5.1 概述

5.1.1 设备管理的任务与功能

1. 设备管理的任务

- (1) 按用户需求提出的要求接入外部设备；
- (2) 尽量提高输入输出设备的利用率。如，发挥主机与外设以及外设之间的真正并行工作能力。

2. 设备管理的功能

(1) 分配设备

按设备的不同类型和操作系统选用的算法分配，包括分配相应的通道、设备控制器以及对未分配的任务或作业进行排队等。

(2) 控制和实现真正的输入输出并行操作

包括通道程序控制、启动设备、及时响应及处理中断讯号等。

(3) 对输入输出缓冲区进行管理

如：逻辑名的管理，多个缓冲区的分时及串并行操作，同类多个外部设备的均衡工作。

(4) 在一些较大系统中实现虚拟设备技术。

5.1.2 发展历史

计算机的基本输入输出设备的发展共经过了三代

- (1) 第一代：键盘和打印机；
- (2) 第二代：鼠标和调制解调器；
- (3) 第三代：手写笔和扫描仪等。

5.1.2 外部设备的分类

在现代计算机系统中，除了 CPU 和内存（也叫主存储器）外，其它大部分硬

件设备都可统称为外部设备。其中包括常用的输入输出设备、外存设备和终端设备等，还包括将外设和主机连接起来的通道(channel)和控制器(controller)。在计算机系统中，从不同角度将设备划分成不同的类型加以管理和调度，归类后简化了设备管理程序，管理工作的关键之一是“分类”和“记录”。

1. 按用户和用户分类

(1) 系统设备(一般是标准设备)

在操作系统的安装时可由用户在安装(install)、设置(setup)或生成(generation)时选择。登记在系统中的标准配置，属于系统的基本配置。如显示器、键盘、鼠标器、光盘驱动器、调制解调器和打印机等。

(2) 用户设备(一般为非标准设备)

在系统生成或摄制是未能登录的非标准设备，通常由用户按需要外加，但需了解操作系统的设备接口设计规程，以便让操作系统对其实施统一的管理和控制。如：实时控制中经常用到的A/D、D/A转换器、图像处理系统中的图像输入输出设备等。

2. 按输入输出传送方式分类(UNIX或Linux操作系统)

(1) 字符设备(character device)

这类设备以字符为单位进行输入输出设备，如终端显示设备和打印机等，在UNIX系统下，用ls -l/dev列目录时，屏幕显示的第一个字符为c。

(2) 块设备(block device)

这类设备是以字符块为单位进行输入输出的设备，在不同的操作系统或同一操作系统的不同版本中，块的大小是不一样的，但在同一操作系统中，块一旦选定，其大小是固定的。一般有512b、1024b或4096b，在UNIX操作系统中，用ls -l /dev列目录时，第一字符为b。

3. 按资源特点分类

(1) 独享设备(monopolize device, independent device)

所谓独享是指这类设备要以用户作业或任务为单位分配，在该用户作业或任务未完成或退出之前，此设备不能分配作其它用。如：打印机、磁带机等。所有字符型设备原则上都是独享设备。

(2) 共享设备(sharing device)

所谓共享，是指多个用户作业、多个任务或多个进程可以同时从这些设备上存取信息，而这里的同时存取实际上是指可以交替地从设备上存取信息。如：软磁盘、应磁盘、光盘等。

(3) 虚拟设备(virtual device)

通过某种软件技术将独享设备改造成多台同类型独享设备或共享设备。如：通过SPOOLing技术可以使一台打印机虚拟成多台打印机。

4. 按设备硬件物理特性分类

(1) 顺序存取设备(sequential access device)

存取时间与物理上当前位置有关，例如：磁带。

(2) 直接存取设备(direct access device)

存取时间与物理上的当前位置关系不大，例如：磁盘。

5. 按设备使用分类

(1) 物理设备(physical device)

一般由操作系统分类命名。如 dos 中的 CON、PRN、LPT、COM 等。

(2) 逻辑设备 (logical device)

用户可以重新命名的设备。

(3) 伪设备 (pseudo device)

泛指具有特定用途的逻辑设备，如模拟的空设备，可以在内存或外存上临时开辟一个区域，充当一个“设备”用。在 dos 中，NUL 就是一个逻辑空设备。如 COPY NUL filename 相当于将 filename 删除。

5.1.4 设备 I/O 方式

1. 询问 (query):

实时系统中必须采用，在一般系统中也广泛使用，但系统开销较大，每次进行输入输出工作，都需要由主机 CPU 干预。

2. 中断 (interrupt)

有硬中断和软中断之分，中断按级别分类排队工作。中断工作主要由中断请求、中断响应和中断服务处理组成。

3. 通道 (channel)

有硬通道和软通道之分，硬通道如典型的 DMA 工作方式；软通道是软件设计中使用的各种“标识”和传送机构等。

4. 直接内存存取方式 (DMA, direct memory access)

DMA 方式有以下几个特点。

(1) 外设和主存之间开辟直接的数据传送通路，如图 5-1。

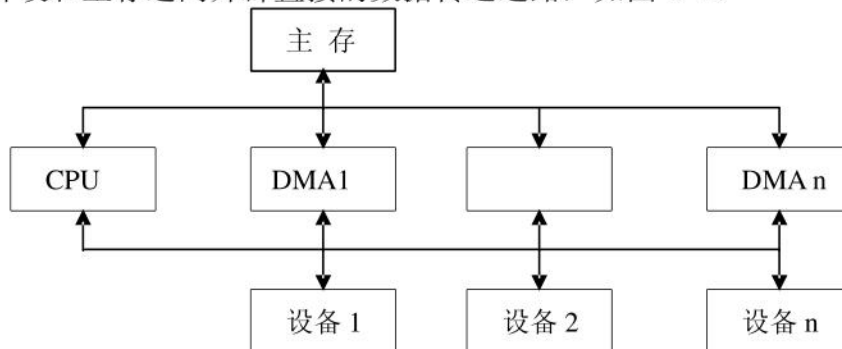


图 5-1 DMA 方式工作图

(2) 使用一个专门的 DMA 控制器 (DMAC)。DMAC 内有控制状态寄存器、传送字数计数器和内存地址寄存器以及可能的数据缓冲器。

(3) 在通常情况下，所有工作周期均用于 CPU 的程序。当设备要输入/输出数据时，窃取一个工作周期，在 DMAC 的控制下，完成外设和内存直接交换数据。这个周期过后，CPU 仍继续执行。DMA 控制实质上取代了 CPU 对 I/O 的控制。

(4) 在输入/输出时，CPU 只需向 DMA 控制器提供必须的控制信息，DMA 控制器类式处理器能自动控制一批信息的传输，传输结束后再向 CPU 发出中断，较好地解决了外设与主机的成批信息交换。

5.1.5 未来展望

1. 未来十年中，要使计算机能看、能说、能听、并与外部设备密切相关；
2. 和目前流行的小闹钟大小相当，甚至小到在身上、衣服上到处可安装；
3. 电脑象电话机一样等待你，计算机联网工作就象打电话、看电视一样方便；

4. 显示器大到能挂在墙上或随意放大缩小，小到能放在提包里或衣袋里；
5. 每台计算机要能连接更多的复杂外部设备，并具有自动的“时空感应”控制功能；
6. 新的连接方式不断出现，有线的和无线的方式，例如 USB、IEEE1394、FireWire 等。
7. 操作系统中，将可能有 90% 的代码支持外设功能；
8. 操作系统可能被“嵌入式”新型数据库查找系统代替；
9. 要求计算机更快更小，特别是适应音频和视频的高速传输，在图形接口方面，如 Intel 的 AGP 技术和 100MHz 新的总线速度、磁盘容量更大。

5.2 外部设备的安装

5.2.1 系统配置

每个系统都有其基本配置，不同的年代，计算机的基本配置也不一样，当前计算机的基本配置有大容量硬盘、高分辨率显示器、光驱、键盘、鼠标和调制解调器等。

5.2.2 用户外接设备

现代操作系统为了方便用户，已在系统中预置了众多知名的设备驱动程序，供用户选用（操作系统的重新生成）。为了节省内存空间，这些众多的设备驱动程序，在用户不配备那些设备是不需要装配入操作系统内核（kernel）的，也就不占用内存空间。只有当用户选用了这些设备是，选用系统提供的添加工具修改系统后，重新构造系统内核，重新启动系统即可使用新的设备。

5.2.3 用户外接特殊设备

用户需要外接的设备，在操作系统中往往没有提供设备驱动程序，特别是一些新产品，以及一些用户需要使用的专用设备，此类设备一般由第三方（设备提供方）提供在不同操作系统下的设备驱动程序，一般有 FOR DOS、FOR WINDOWS、FOR UNIX 等。

5.2.4 外部设备的即插即用

即插即用—plug and play (PnP) 是目前外设的一种新的外设安装方式。它不需要用户来确定外设对系统资源（中断、地址和通道）的占用，而由系统自动地进行分配。它首先使用在 Windows 操作系统中。

5.2.5 设备驱动程序的编制

在不同的操作系统中所采用的设备处理方式并不完全相同，书写的格式也不一样。一般有以下几种方法：

1. 不纳入操作系统的“外挂式”，可直接利用 CPU 提供的机器指令编写，也可利用高级语言提供的相关语句编写，或利用串行口及并行口的标准接口程序“替换代入”控制设备。
2. 利用操作系统提供的系统调用或程序请求，控制外部设备的三个基本寄存器：设备状态寄存器、数据寄存器和控制寄存器的各位内容。
3. 要纳入操作系统时，要了解该操作系统设备处理程序的表格规范和分段格式等详细内容。
4. 一般外购设备，不论是第三方开放、或是操作系统协作方提供的，都带有适合该操作系统的设备处理程序。
5. 有时出来的一些新产品，可以直接到该公司提供的网站上去找新的驱动程序

或升级软件。

5. 2. 6 CPU 与外部信息的交换

设备和 CPU 之间的数据传送控制方式有四种，分别为程序直接控制方式、中断控制方式、DMA 方式和通道方式。

1. 各种控制方式的比较

程序直接控制方式和中断控制方式只适用与简单的，外设很少的计算机系统，因为程序直接控制方式耗费大量的 CPU 时间和无法检测发现设备或其它硬件产生的错误，而且设备和 CPU、设备和设备之间只能串行工作。中断控制方式虽然在一定程度上解决了上述问题，但由于中断次数多，因而 CPU 仍需要花大量的时间来处理中断，而且中断次数的增多也限制了并行工作的外设的台数，及导致数据丢失等问题。DMA 和通道技术比较好地解决了上述问题。这两种方式采用了外设和内存直接交换数据的方式。只有在一段数据传送结束时，这两种方式才发出中断信号要求 CPU 做善后处理，从而大大减少了 CPU 的工作负担。DMA 和通道方式的区分是，DMA 方式要求 CPU 执行设备驱动程序启动设备，给出存放数据的内存始址以及操作方式和传送字节长度等；而通道控制方式则是在 CPU 发出 I/O 启动命令之后，由通道指令来完成这些工作。

2. CPU 与外部设备的通讯技术

(1) 串行通讯和并行通讯：并行通讯，数据所有位同时传送，速度快；串行通讯的数据位逐位顺序传送，长距离传送时节省传输线，但传送速度慢。

(2) 异步通讯 (asynchronous data communication) 和同步通讯 (synchronous data communication)

异步通讯方式一般以一个字符为一帧，一帧最少由三部分组成：起始位、数据、停止位。

同步通讯方式将异步通讯方式中的起始位和停止位去掉，发送端和接受端用一同步脉冲进行位同步。

(3) 各种编码技术

计算机系统常用的编码有 ASCII 码、BCD 码、EBCD 码，EBCDIC 码等，后几种编码主要用在中大型机上。

5. 2. 7 总线和接口

1. PC 系统总线

PC 系统总线见表 5-1

名 称	性 能	开发者	备 注
S100	双排 50 引脚，间距 0.1 英寸，8080CPU。	IEEE696 标准	
PC/XT	62 引脚，4.77MHz，2.38MB/s。		
ISA (PC/AT) (industry standard architecture)	62+36 引脚，8MHz，为 16 位 286/AT 开发的 AT 标准。	IBM。之后为 IEEE 标准	广泛使用
MCA (micro channel)	10MHz，40MB/s，PS/2 微通道总线。	IBM	未能推广

architecture)			
EISA(extended industry standard architecture)	专为 32 位多 CPU 扩展总线标准, 向下兼容 ISA, 33MB/s	EISA 集团 (Compaq 、 HP、 AST 等)	
VISA(video electronics standards association)	40MHz, 132MB/s, 486 以上开放总线, 32 位。	VESA 组织 (IBM 、 Compaq 、 AST 、 HP 等)	目 前 已 淘 汰
PCI(peripheral component interconnect)	插入 CPU 与原总线之间的一级总线。	Intel 联合 IBM、 Compaq 、 AST 、 HP 等	目 前 广 泛 使用
AGP(accelerated graphics port)	显示卡与主存之间, 提高三维图形处理能力; 第一代 (1X) 66MHz 总线频率, 266MB/s 数据传输速率; 第二代 (2X) 133MHz, 数据传输率 533MB/s。		

表 5-1 PC 系统总线结构简表

2. 标准接口类型

标准接口类型见表 5-2

接口名称	性 能	开发者	备注
IDE/EIDE (IDE: integrated devices electronics)	40 线 EIDE: 13.3MB/s, 支持 NORMAL、LBA 和 LARGE 三种硬盘工作模式	Compaq Western Digital	广 泛 应用
Ultra DMA33/66	Ultra DMA66 (或 Ultra ATA-66) :80 针、66.6MB/S	Quantum、Intel	
SCSI(small computer system interface)	系统级接口/总线, 50 芯, 18 条 母线 SCSI-1 (4.0MB/s)、 SCSI-2 (10.0MB/s) Ultra SCSI 的 Ultra160/m:160MB/s	ANSI X3T9.2 定义标准	广 泛 使用
USB(Universal Serial Bus)	统一接口* 传 输 速 率 为 1.5/12/120/240Mbps	Intel 等七家 公司	已 开 始 推 广
IEEE 1394	串行接口 / 总线 , 原型为	Apple Mac 机	

	FireWire。之后由 IEEE 规范		
Device Bay	创建新 PC。主板仅 CPU，其它都 外接，包括电视、电话等家电。	Microsoft 、 Intel、Compaq	可 能 搁浅！

表 5-2 标准接口类型简表

注：USB 接口是美国 Intel 公司开发的新型接口技术，由具备 USB 的 PC 计算机系统、支撑软件和接口设备三部分组成。理论上讲，一个 USB 接口可以同时接入 127 个设备。其特点是使用方便、速度快、连接灵活、独立供电和支持多媒体。

5.3 输入输出设备分配算法

(1) 先来先服务 (first come first service, FCFS)

每一设备有一队列管理，当多个作业或任务对同一设备提出 I/O 请求时，该算法把请求的先后次序排成一个等待该设备的队列，设备分配程序把设备分配给队列中的第一个作业。

(2) 按优先级 (Priority)

一般是系统高于用户，超级用户（系统管理员）高于一般用户。对于优先级相同的 I/O 请求，则按先来先服务的原则分配。

5.4 设备管理技术

5.4.1 磁盘管理技术

1. 磁盘简介

(1) 磁盘有固定头磁盘和活动头磁盘两种。固定头磁盘是每个磁道都有一个磁头，这种磁盘的磁道转换快，但价格较贵。一般情况是一个磁道只有一个磁头，此磁头在径向移动以存取不同磁道上的信息，它需要磁头控制部件支持，但造价较低。

(2) 存取磁盘信息需要以下的地址信息：驱动器号、盘面号、磁道号、磁头数。

(3) 磁盘的存取速度由三部分组成：磁头定位时间、扇区定位时间和实际信息传输时间。

(4) 对于每一个磁盘驱动器，都有一个 I/O 请求队列，每当一个进程需要 I/O 时，它发出一个系统调用给操作系统，I/O 请求包含以下必要的信息：(1) 输入或是输出；(2) 磁盘地址；(3) 内存地址；(4) 传送信息长度。如果所要求的磁盘驱动器和控制器是空闲的和可用的，则 I/O 请求被立即执行；如果设备和控制器正在为其它进程的 I/O 请求服务，则 I/O 请求必须排队等待。

2. 硬盘容量讨论

(1) 硬盘的存取过程

操作系统（或特殊的应用程序绕过操作系统）通过 INT 13 调用 BIOS 中的磁盘服务程序，该服务程序再将 INT 13 请求转换为硬盘的 ATA 接口请求，继而在硬盘上进行数据存取。与硬盘容量有关的部分就是 INT 13 接口和 ATA 接口。INT13 接口程序保存在主板的 BIOS 中，而 ATA 接口与硬盘驱动器有关，这两种接口都是通过寄存器来寻址的。

(2) 硬盘寻址的两种方式

(a) CHS 方式

CHS 方式是通过 cylinder-柱面数，head-磁头数，sector-每柱面扇区数来寻址的，其支持硬盘的最大容量为：

$$1024 \times 63 \times 16 \times 512 / 1024 \text{KB} = 504 \text{MB}$$

(b) LBA 方式

LBA-logical block address (逻辑块地址寻址法), LBA 寻址方式, 简单的说, 就是把原来分别表示 C、H、S 的三个参数的 28 个位 (16+4+8), 看成一个表示地址的数, 因而可以寻址的最大空间为:

$$2^{28} \times 512 / 1024 = 134\text{G}.$$

5. 4. 2 缓冲技术

1. 在 I/O 部分设置缓冲的理由

(1) 解决信息的到达率和离去率不一致的矛盾

由于输入数据的速度和处理数据的速度往往不相同, 于是, 有必要设置一个缓存, 作为它们之间的缓冲, 以解决它们速度上的匹配以及由于速度不一致而造成数据的丢失和紊乱。

(2) 缓存起中转站的作用

缓存可以是外存与外存之间的中转站

(3) 使得一次输入的信息能够多次使用

这主要是用在文件系统中, 由于有的文件是可以共享的, 所以会出现多个任务同时需要使用一个文件的情况, 这样就可以从缓存中读取所需内容, 而不必启动 I/O 到外存上去读取; 另外, 在通道或控制器内设置局部寄存器作为缓冲寄存器, 可暂存 I/O 信息, 以减少中断 CPU 的次数。

2. 缓冲的种类及特点

缓冲缓冲技术管理按缓冲存储器分为单缓冲、双缓冲、多缓冲和缓冲池。

(1) 单缓冲

单缓冲是在设备和处理机之间设置一个缓冲器 (区)。设备和处理机交换数据时, 先把被交换数据写入缓冲器 (区), 然后, 需要数据的设备或处理机从缓冲器中取走数据。由于缓冲器属于临界资源, 即不允许多个进程同时对一个缓冲器操作, 因此, 尽管单缓冲能匹配设备和处理机之间的处理速度, 但是, 设备和设备之间不能通过单缓冲达到并行操作。

(2) 双缓冲

解决两台外设、打印机和终端之间的并行操作的办法是设置双缓冲, 也就是设置两个缓冲器, 分别用于外设的输入和输出。双缓冲仅仅是一种说明设备和设备之间并行操作的一个简单模型, 并不能用于实际系统中的并行操作。

(3) 多缓冲和缓冲池

多缓冲是把多个缓冲区连接起来组成两部分, 一部分专门用于输入, 另一部分专门用于输出的缓冲结构。缓冲池则是把多个缓冲器连接起来统一管理, 既可用于输入又可用于输出的缓冲结构。

在缓冲池管理中, 一个缓冲区由两部分组成: 一部分用来标识该缓冲器和用于管理的缓冲首部, 另一部分是用于存放数据的缓冲体。这两部分有一一对应的关系。对缓冲池的管理是通过对每一个缓冲器的首部进行操作实现的。缓冲首部包括设备号、设备上的数据块号 (块设备时)、互斥标识位以及缓冲队列连接指针和缓冲器号等。

(4) 缓冲的实现方式

(a) 硬件方式

一般是采用专用硬件缓冲器，如 I/O 控制器中的数据缓冲寄存器等。

(b) 软件方式

在内存中开辟出一个具有 N 个单元的专用缓冲区，以便存放输入输出数据。操作系统为每一个缓冲区建立一个数据结构，称为缓存控制块 BCB (buffer control block)。操作系统通过 BCB 对每一个缓存实施具体的管理。

5. 4. 3 虚拟设备技术

1. SPooling—simultaneous peripheral operations on-line，即外部设备联机并行操作，它是慢速字符设备如何与计算机交换信息的一种技术，通常又称为“假脱机技术”。这是一种预输入、缓输出和转储的管理技术。

2. 字符设备和主机交换信息的两种方式

有两种基本方式，即脱机方式和联机方式。

(a) 联机方式，就是一般字符设备直接与内存交换信息。这种交换信息的缺点是：由于字符设备信息传送速度慢，当一个任务要求从外设上输入信息时，往往要等候很长的时间；其次，由于字符设备本质上是顺序存取设备的，因此，在一个任务的信息未传输完之前，别的任务不能进行输入或输出，这就降低了整个系统的并行处理能力。

(b) 脱机方式，以输入为例，先使用一台外围机把字符设备上的信息输入到块设备上（磁带或磁盘等），然后把块设备与主机联上。这样，主机在从块设备上（而不是从字符设备）上输入它所需要的信息。这种输入方式的优点是可以提高输入的并行度，缺点是需要人工干预，这就会降低系统的平均运行速度，而且容易出差错和损坏设备。

(c) SPooling 技术是对上述两种信息传输方式的综合，它取两者的长处，而避免两者的短处，以联机的输入、输出方式，获得脱机的输入、输出的优点，所以 SPooling 技术有时也叫“假脱机技术”。

3. SPooling 系统的组成

1. 输入井和输出井

这是在磁盘上开辟的两大存储空间。输入井是模拟脱机输入时的磁盘，用于收容外部设备 (I/O) 设备输入的数据。输出井是模拟脱机输出时的磁盘，用于收容用户程序的输出的数据。

2. 输入缓冲区和输出缓冲区

在内存中要开辟两个缓冲区：输入缓冲区和输出缓冲区。输入缓冲区用于暂存由输入设备送来的数据，以后再传送到输入井。输出缓冲区用于暂存从输出井送来的数据，以后再传送给输出设备。

3. 程序工作的输入进程 SPi 和输出进程 SPo

进程 SPi 模拟脱机输入时的外围控制机，将用户要求的数据从输入机通过输入缓冲区再送到输入井。当 CPU 需要输入数据时，直接从输入井读入内存。SPo 进程模拟脱机输出时的外围控制机，把用户要求输出的数据，先从内存送到输出井，待输出设备空闲时，再将输出井中的数据，经过输出缓冲区送到输出设备上。

4. SPooling 系统的特点

- (1) 提高了 I/O 速度
- (2) 将独占设备改造为共享设备

在 SPooling 系统中，实际上并没有为任何进程分配，而只是在输入井和输出井中，为进程分配一存储区和建立一章 I/O 请求表。这样，便把独占设备改造为共享设备。

- (3) 实现了虚拟设备功能

宏观上，虽然是多个进程在同时使用一台独立设备，而对每一个进程而言，它们都认为是独占了一个设备。当然，该设备只是逻辑上的设备。SPooling 系统实现了将独占设备变换为若干台对应的逻辑设备的功能。

5. 4. 4 共享打印机

共享打印机是 SPooling 技术应用的典型实例。它将一台打印机改造为供多个用户共享的打印机，目前广泛地应用在多用户系统和局域网中。

1. 打印机共享的 SPooling 技术

SPooling 系统在收到用户进程的打印请求时

- (1) 由输出进程在输出井中申请一个空闲盘块区，并将要打印的数据送入其中；
- (2) 输出进程再为用户进程申请一张空白的用户请求打印表，并将用户的打印要求添入其中，再将该表挂到请求打印队列上。(3)如果还有进程要求打印输出，系统仍可接受其请求，也同样为该进程重复 (1)、(2)。

2. 打印队列

请求打印队列是由若干张请求打印表所形成的队列，系统为每个请求打印的进程建立一张请求打印表。如果打印机空闲，输出进程将从请求打印队列的对首取出一张请求打印表，根据表中的要求将要打印的数据从输出井传送到内存缓冲区，再由打印机进行打印。打印完成后，输出进程再查看请求打印队列中是否还有等待打印的请求表。若有，又取出第一张表，并根据其中的要求进行打印，如此下去，直至请求队列为空为止，输出进程才将自己停止工作，直到下次再有打印请求是才被唤醒继续工作。

5. 5 设备处理程序编制内容

5. 5. 1 设备处理程序

设备处理程序通常又称为设备驱动程序，它是 I/O 进程与设备控制器之间的通信程序，又由于它常以进程的形式存在，故以后就简称为设备驱动进程。其主要任务是接收上层软件发来的抽象要求，例如，read, write 等命令，再把它转换为具体要求后，发给设备控制器，启动设备去执行。此外，它也将由设备控制器发来的信号传送给上层软件。一般设备驱动程序包含五部分：

- (1) 设备标题 (device header) ；

- (2) 数据存储和局部过程 (data storage & local procedure) ;
- (3) 策略过程 (strategy procedure) ;
- (4) 中断过程 (interrupt procedure) ;
- (5) 命令处理子程序 (command processing) ;

5. 5. 2 设备驱动程序的功能

- (1) 将接收到的抽象要求转换为具体要求;
- (2) 检查用户 I/O 请求的合法性, 了解 I/O 设备的状态, 传递有关参数, 设置设备的工作方式;
- (3) 发出 I/O 命令, 启动分配到的 I/O 设备, 完成指定的 I/O 操作;
- (4) 及时响应由控制器或通道发来的中断请求, 并根据其中断类型调用相应的中断处理子程序进行处理;
- (5) 对于设置有通道的计算机系统, 驱动程序还应能够根据用户的 I/O 请求, 自动地构成通道程序。

5. 5. 3 设备驱动程序的特点

设备驱动程序和一般的应用程序或系统程序相比较, 存在如下一些特点:

- (1) 驱动程序主要是在请求 I/O 的进程与设备控制器之间的一个通信程序。它将进程的 I/O 请求传送给控制器, 而且把设备控制器中所记录的设备状态、I/O 操作完成情况, 反映给请求 I/O 的进程。
- (2) 驱动程序与 I/O 设备的特性紧密相关。因此, 对于不同类型的设备, 应配置不同的驱动程序。
- (3) 驱动程序与 I/O 控制方式紧密相关。常用的设备控制方式是中断方式和 DMA 方式。
- (4) 由于驱动程序与硬件紧密相关, 因而其中的一部分程序必须用汇编语言编写, 目前有很多驱动程序, 其基本部分已经固化, 放在 ROM 中, 例如, DOS 操作系统中的 BIOS。

5. 5. 4 设备处理方式

设备驱动程序的处理过程对不同类型的设备应有不同的设备驱动程序, 但大体上它们都可以分成两部分, 它除了需要有能够驱动 I/O 设备工作的驱动程序外, 还需要有设备中断处理程序来处理 I/O 完成后的工作。

1. 设备处理程序的处理过程

- (1) 将抽象要求转换为具体要求

通常在每个设备控制器都有若干个寄存器, 它们分别用于暂存命令、数据和参数等。用户及上层软件对设备控制器的具体情况毫无了解, 因而只能向它们发出抽象的要求。因此, 就需要将这些抽象的要求转换为具体要求。例如, 将盘块号转换为磁盘的盘面、磁道号及扇区。这一转换工作就由设备驱动程序来完成。

- (2) 检查 I/O 请求的合法性

对于任何输入设备都只能完成一组特定的功能, 如该设备不支持这次 I/O 请求, 则认为这次 I/O 请求非法。例如, 用户试图请求从打印机输入数据, 或是在一个打开时规定为读的设备上进行写操作, 则用户的请求将被拒绝。

(3) 读出和检查设备的状态

要启动某个设备进行 I/O 操作，其前提条件应是该设备正处于空闲状态。因此，在启动设备之前，要从设备控制器的状态寄存器中读出设备的状态。例如，为了向某设备写入数据，此时应先检查该设备的状态是否处于接收就绪，只有它处于接收就绪状态时，才能启动其设备控制器，否则只能等待。

(4) 传送必须的参数

还需传送必须的参数，例如，在启动磁盘进行读写之前，应先将本次要传送的字节数、数据应到达的内存始址送到控制器相应的寄存器中。

(5) 方式的设置

有些设备有多种工作方式，如通过 RS-232 进行异步通信。在启动设备之前，应规定下述参数：波特率、奇偶校验方式、停止位数目及数据长度等。

(6) 启动 I/O 设备

在完成上述各项准备工作后，驱动程序可以向控制器中的命令寄存器传送相应的控制命令。对于字符设备，若发出的是写命令，驱动程序将把一个数据传送给控制器，若发出的是读命令，则驱动程序等待接收数据，并通过从控制器的状态寄存器读入状态字的方法来确定数据是否到达。驱动程序发出 I/O 命令后，基本的 I/O 是在设备控制器的控制下完成的。

2. 设备处理方式

(1) 为每一类设备设置一个进程，它专门执行这类设备的 I/O 操作。

(2) 在整个系统中设置一个 I/O 进程，专门负责对系统中所有各类设备的 I/O 操作。也可以设置一个输入进程和一个输出进程，分别负责处理系统中所有的各类设备的输入或输出操作。

(3) 不设置专门的设备处理进程，而是只为各类设备设置相应的设备处理程序（模块），供用户进程或系统进程调用。

5.5.5 设备驱动程序举例

设备驱动程序是驱动物理设备和 DMA 控制器或 I/O 控制器等直接进行 I/O 操作的子程序的集合。它们负责设置相应设备有关寄存器的值，启动 I/O 设备进行 I/O 操作，指定操作的类型和数据流向等。

1. DOS 系统设备驱动程序

(1) DOS 在 BIOS 模块中已提供了控制标准硬件设备的常驻设备驱动程序。

(2) 系统中设置有设备开关表 DST。设备开关表中给出相应设备的各种操作子程序的入口地址，例如，打开、关闭、读、写和启动设备子程序的入口地址。设备开关是二维结构，其中的行和列分别表示设备类型和驱动程序类型。I/O 控制过程为进程分配设备和缓冲区之后可以使用设备开关表调用所需的设备驱动程序进行 I/O 操作。

(3) DOS 为设备驱动程序选定的扩展名为 .BIN 或 .SYS，在 DOS 的系统盘的 config.sys 中插入如下命令：

DEVICE=[d:][path]文件名.SYS 进行设备驱动。

2. Windows 系统的设备管理

Windows 系统的特点

- (1) 在 MS-DOS 的基础上发展而来。
- (2) 吸收了“树型目录”和设备文件统一管理的优点，向“文件夹”发展。
- (3) 支持外部设备的即插即用 (PnP) 功能，在*.inf 文件中描述安装信息和在*.vxd 文件中具备虚拟设备驱动程序等。
- (4) Windows 中的动态连接库 (dynamic link library, DLL)

在 MS-DOS 环境下，所有程序的目标组件在创建过程中都被静态连接起来，而 Windows 则允许动态的连接，即一些特定结构的库可以在执行过程中被装入和连接。DLL 是一种基于 Windows 的程序模块，它可以在运行时刻被装入和连接。许多应用都被分为一系列的主程序和 DLL。在 Windows 中，多个应用可以共享同一个 DLL，这就可以大大节省内存和磁盘空间，同时动态连接还可以大大增强对程序调试的灵活性，因为人们可将动态连接库单独编译和调试。

3. UNIX 中设备管理

UNIX 设备管理程序主要分为三部分：最底层的设备驱动程序、缓冲区管理和最上层与用户相“见”的特殊文件处理。

设备驱动程序包含 I/O 队列管理、中断管理、地址转换、缓冲区管理和最上层与用户相“见”的特殊文件处理。其最大的特点是设备寄存器与内存地址一起编址，是从内存的最高地址开始，在核心装入内存时，从内存最高地址存放实际的设备驱动程序；缓冲区管理包括缓冲区申请、查链、摘链、清零和释放缓冲区等；按 UNIX 文件系统统一格式处理，设备处理文件也做成特殊文件，放在/dev 目录下，这里仅仅是设备文件名与内码 I 节点的对应关系，包含用户 I/O 请求，命令合法性检查和参数处理等。

5. 6 重点小结

1. 设备管理的主要任务是使用户不必了解具体设备的物理特性，能方便于设备的扩充和更新，能为用户提供一个透明的、易于扩展的通用接口，控制设备和 CPU 之间进行 I/O 操作。另外，还要尽可能地提高设备和设备之间、设备和 CPU 之间的并行操作度以及设备利用率。

2. 本章从设备分类出发，对设备和 CPU 之间的数据传送的控制方式、中断和缓冲技术、设备分配原则和算法、I/O 控制过程以及设备驱动程序进行了介绍和讨论。

3. 设备分配的原则和算法。设备分配应保证设备有高的利用率和避免产生死锁。进程只有在得到了设备、I/O 控制器和通道（通道控制方式）之后，才能进行 I/O 操作。另外，用户进程给出的 I/O 请求中包含逻辑设备号，设备管理程序必须将其转换为实际的物理设备，I/O 请求命令中的其它参数将被用来编制通道指令程序或由设备开关表选择设备驱动程序。

4. I/O 控制过程是对整个 I/O 操作的控制，包括对用户进程 I/O 请求命令的处理、进行设备分配、缓冲区分配、启动通道指令程序或驱动程序进行真正的 I/O 操作，以及分析中断原因和响应中断等。

第 6 章 进程及处理机管理

现代操作系统的重要特点是程序的并发执行，及系统所拥有的资源被共享和系统用户随机地使用系统。采用一个什么样的概念，来描述计算机程序的执行过程和作为资源分配的基本单位才能充分反映操作系统的执行并发、资源共享及用户随机的特点呢？这个概念就是进程。

6. 1 概述

6. 1. 1 操作系统核心的功能和特点

1. 进程与操作系统的关系：五大功能之一

(1) 高级（宏）处理机管理

即作业调度，确定系统中哪些作业将获得 CPU；

(2) 低级（微）处理机管理

即进程调度，确定系统中哪个作业中的哪个进程将获得 CPU。

2. 什么是进程？

(1) 进程的定义

进程是一个具有一定功能的程序关于某个数据集合的一次运行活动。

进程是操作系统动态执行的基本单元，在传统的操作系统设计中，进程既是基本的分配单元，也是基本的执行单元。

(2) 进程划分的原则

进程大小的“分割”设计，因不同的操作系统设计者而异。进程分得太大，极端情况就变成顺序执行的计算机，也就失去了并发性，也就降低了系统资源；但另一极端，进程分得太小，CPU 为多个用户或一个用户的多个任务服务时，开销急剧增大。因为，在进程间的时空转换及工作量将大大增加。

3. 操作系统核心功能

(1) 调度进程，决定哪个进程运行、挂起、交换等；

(2) 分配内存，哪个进程得到内存；

(3) 管理和控制文件系统；检查“许可证”、修改目录、路径等；

(4) 处理系统调用：由用户的进程发“请求”，系统根据资源的充分利用，统筹安排；

(5) 处理输入输出的请求和工作。

总之，操作系统的五大功能都必须由核心负责协调工作。

4. 操作系统核心的形式

(1) 常驻内存：计算机启动后，操作系统核心常驻在内存

(2) 操作系统核心是一组服务功能程序的集合，它由许多可执行的工作模块装配而成。操作系统中大量使用表格数据结构。通过大量内部表格内容的组合并发协调执行，大量工作是查表、修改和维护表格；

(3) 操作系统设计有两种观点，即用户观点和资源观点。工作时有两大类表格：系统态和用户态。一类面对用户的“订单”，另一类由系统内部管理分工决定。

6. 1. 2 为什么要引进进程概念

引入进程的概念，关键是共享资源引起的。在顺序执行模块的程序中，有如下特点：

(1) 封闭性 (closure property)；

(2) 可再现性 (re-appearable)；

(3) 调试容易；

(4) 设备利用率不高。

6. 1. 3 顺序执行与并发执行

引入进程的关键是资源共享，而从资源的观点看，有效管理共享资源是计算机操作系统的最重要内容。

顺序执行与并发执行见表 6-1。

顺序执行	并发（共行）执行
程序顺序执行	间断执行，多个程序各自在“走走停停”中进行
程序具有封闭性	程序失去封闭性
独享资源	共享资源
具有可再现性	失去可再现性
	有直接和间接的相互制约

表 6-1 顺序执行与并发执行比较

6. 2 进程的定义和特征

在任务执行过程中切割成独立的单元涉及到进程(process)的组成内容、任务激活(active)以及线程(thread)。线程是近年来由进程发展而来，一般定义为程序执行中单个顺序的流控制，比进程优越之处是执行中占有相同的内存空间。

6. 2. 1 程序与进程

1. 程序与进程的对比

程序与进程的对比见表 6-2

程 序	进 程
静态的指令序列描述	动态的程序执行过程
永久性软件资源	动态生存的暂存性资源
工作时一个程序可以由多个进程在工作	一个进程在工作至少对应有一个程序
操作系统管理下，经用户态由系统态系统调用执行	由操作系统核心在内部进行分配调度

表 6-2 程序与进程的对比表

2. 程序与进程的类比

程 序	进 程
唱歌的曲谱或音乐乐器的乐谱	演出或演奏
剧本	演出
菜谱	烹调

表 6-3 程序与进程的类比

6. 2. 2 进程的五個基本特征

(1) 动态性

进程是程序在并发系统的一次执行，一个进程有一个从产生到消失的生命期；

(2) 并发性

正是为了描述程序在并发系统内执行的动态特征才引入了进程，没有并发就没有进程；

(3) 独立性

每个进程的程序都是相对独立的顺序程序，可以按自己的方向和速度独立地向前推进；

(4) 制约性

进程之间的相互制约，主要表现在互斥地使用资源和相关进程之间必要的同步和通讯；

(5) 结构性

进程=PCB（进程控制块）+程序+数据集合。

6. 2. 3 进程与线程

1. 线程的定义

简单地讲，进程就是程序的一次执行过程。而线程是由进程派生出来的一组代码（指令组）的执行过程。

2. 线程的特点

线程是由进程派生出来的，一个进程可以产生多个线程，线程的特点是共享进程的内存空间，它们可以并发、异步地执行。

3. 采用线程的优点

- (1) 使同一个程序能有几个并发执行的路径，提高了执行速度；
- (2) 线程需要的开销比进程小。

4. Windows 的多任务调度

(1) Windows 3.x 采用协作式多任务

在 Windows 3.x 中，实行协作式的多任务方式，多个应用程序之间必须相互协调，依次实现操作系统的管理和控制。它并不是真正的多任务执行，为了让操作系统把控制权从一个程序转移到另一个程序，当前活动的程序就必须周期地检查一个消息队列。如果一个程序不能检查消息队列，操作系统就不能实现控制权的转移。

(2) Windows 9x 采用抢先式多任务

(a) 操作系统可以在需要时中断当前任务，再按照任务队列中各个任务的优先级来进行多任务的调度。为兼容起见，基于 Windows 的 16 位 (Win16) 应用程序仍采用协作式；来完成多任务的执行。

(b) 在 Windows 9x 中，抢先式多任务的执行实际上就是抢先式多线程的实现。每个线程有一个优先值，范围从 0 到 31，数字越大，则优先级越低。

(3) 在抢先式多任务中，基于 Win32 的应用程序不必让位给其它程序就能以友好的方式实现多任务。操作系统会根据系统的需要把控制权交给一个运行中的任务，或从一个运行中的任务移走控制权。

(4) 在 Windows 9x 中，此种调度方式可能是系统不能稳定运行的原因之一。

6. 3 进程调度

6. 3. 1 进程的描述

1. 多道程序并发执行的特点

- (1) 资源分配的动态性

多道程序在运行中可以提出分配资源的请求，操作系统酌情满足它，这就带来了资源分配的动态性。

(2) 程序执行的间断性

在多道程序系统中，多个程序共享一个或多个 CPU，一个程序在运行一段时间后便要让位给另一程序运行。因此，每个程序的运行都是处于“走走停停”的状态，这就是程序执行的间断性。

(3) 程序间的通讯

如果多个程序之间有一种合作关系，例如共同求解一个题目，则它们在运行过程中就有可能互相传递数据，这就是程序间的通讯。

(4) 程序间的同步和互斥

有合作关系的程序不仅要互相通讯，而且可能要调整它们之间的相对速度，比如一个程序必须等待另一个程序的结果才能继续运行。这就是程序间的同步。而由于多道程序系统中的资源共享，程序段之间对共享资源的竞争而导致了程序段之间的互斥。

2. 进程的引入和构成

(1) 进程的引进

上面所列的多道系统中的程序运行的新特点，程序本身是无法描述的，为此，当一个程序在并发系统中执行时，需引进一个新的数据结构来记录和描述这些特征。这样，新引进的数据结构与它所描述的程序便形成了一个有机体。这个有机体就称为进程。

(2) 进程的构成

进程=PCB+程序+数据

其中，PCB (process control block) 为记录程序在并发系统中执行时的动态特性的数据结构。

3. 程序和进程的形象比较

- (1) 两个施工队按同一图纸在两个不同的地方建房，这是两个不同的进程；
- (2) 同一施工队按同一图纸在不同时间多次建房，这是多个进程。

由此可见，进程是一个与具体的时间和空间有关的动态概念。

4. 进程的定义和特征

到目前为止，进程有多种定义，如：

- (1) 进程是程序的执行；
- (2) 并行程序称为进程；
- (3) 进程是可以和别的计算并发的计算；
- (4) 进程是一个数据结构及在其上进行操作的程序。

这些定义都从不同的侧面描述了进程的特征，都一定的道理，但我们认为下面的定义更全面和更准确：

进程是程序在一个数据集合上运行的过程，它是传统操作系统进行资源分配和调度的一个独立单位。此定义包含有如下的含义：

- (1) 进程是一个动态的概念，而程序是一个静态的概念；
- (2) 进程包含了一个数据集合和运行其上的程序；

(3) 同一程序运行于若干不同的数据集合上时，它将属于若干个不同的进程，或者说，两个不同的进程可包含相同的程序；

(4) 系统分配资源是以进程为单位的，所以只有进程才可能在不同的时刻处于几种不同的状态，即等待、就绪、运行。

(5) 从微观上看，进程是轮换地占有处理机而运行的，从宏观上看，进程是并发地运行的。

6.3.2 进程的状态及转换

1. 进程的状态

一般来说，一个进程在它的生命周期有三种基本状态，分别为就绪(ready)、执行(execute)和等待(waiting)。一个进程在刚创建初期处于“进入”状态，在运行终止后处于“完成”状态。

- (1) 就绪——进程具备运行条件，但尚未占用 CPU；
- (2) 执行——进程正在占用 CPU；
- (3) 等待——进程由于等待 某一事件不能享用 CPU。

进程的三种基本状态及关系如图 6-1

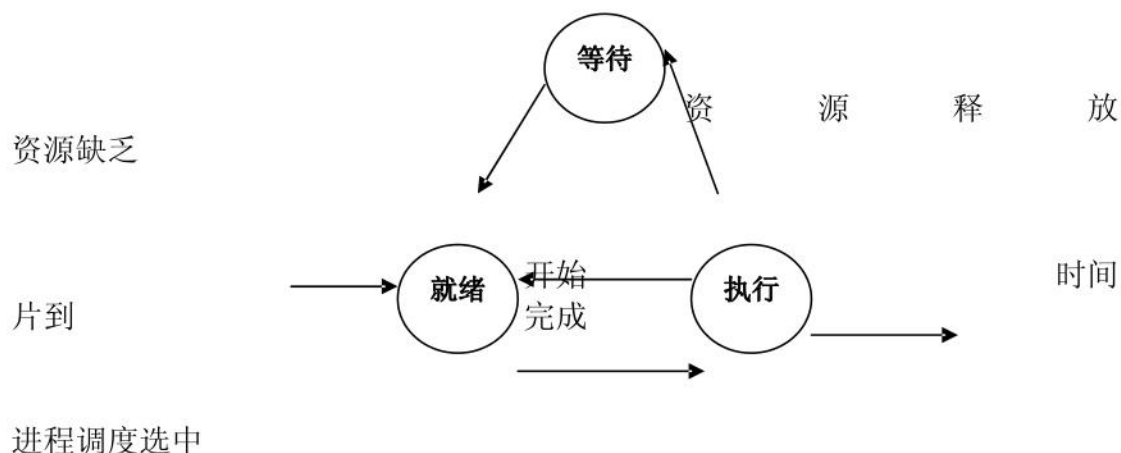


图 6-1 进程的三个基本状态示意图

2. 引起进程状态转换的原因

(1) CPU 调度（低级调度）：CPU 调度按某种原则从就绪队列中调度一个进程到 CPU 上运行，该进程就从就绪状态变为运行状态；与此同时，原运行进程从运行状态变为就绪状态。因此，这两种状态变化是同时发生的。

(2) 进程在运行过程中需要等待某一事件，例如，等待分配某一资源，等待 I/O 操作完成等。一个进程在需要等待某一事件时主动退出 CPU，并使自己处于阻塞状态，引起状态变化。

(3) 如果进程所等待的事件发生了变化，例如，一次 I/O 完成了，于是进程便被解除阻塞状态，变为就绪状态。

(4) 一个具体的进程在任何一个指定的时刻必须而且只能处于一种状态。

3. 进程状态转换的说明

(1) 进程之间的状态转换并非都是可逆的

进程既不能从等待变为运行，也不能从就绪变为等待。

(2) 进程之间的状态转换并非都是主动的，在很多情况下都是“它动的”事实上，只有运行到等待的转换是进程的主动行为（主动调用调度管理程序），其它都是它动的，例如，从执行到就绪，通常是时钟中断引起的，从等待到就绪，是一个进程把另一个进程唤醒。

6. 3. 3 进程控制

1、进程控制的任务

进程控制的任务就是系统使用一些具有特定功能的程序段来创建、撤消进程及完成进程各状态间的转换，从而达到多进程高效率并发执行和协调、实现资源共享的目的。

2、原语

一般，我们把系统态下执行的某些具有特定功能的程序段成为原语，原语可是机器指令级的扩充，其特点是执行期间不允许中断、它是一个不可分割的基本单位。它们都在系统态下执行，且都是为了完成某个系统管理所需要的功能和被高层软件调用。

在操作系统中，一般都把进程控制用的程序段做成原语，用于进程控制的原语有：

创建原语、撤消原语、阻塞原语、唤醒原语等。

3、进程的创建与撤消

(1) 进程的创建

进程创建的方式有两种：

(a) 由系统程序模块统一创建。例如在批处理系统中，由操作系统的作业调度程序为用户进程创建相应的进程以完成用户作业所要求的功能。

(b) 由父进程创建。例如在 UNIX 操作系统中，父进程创建子进程以完成并行工作。

(c) 创建原语的流程图如图 6-a

(2) 进程撤消

(a) 进程撤消的原因

1) 该进程已完成所要求的功能而正常终止。

2) 由于某种错误导致非正常终止。

3) 祖先进程要求撤消某个子进程。

(b) 撤消原语的流程如图 6-b

4、进程的阻塞与唤醒

(1) 进程的阻塞

阻塞原语在一个进程期待某一事件发生但发生条件尚不具备时，被该进程自己调用来阻塞自己。阻塞原语在阻塞一个进程时，由于该进程正处于执行状态，故应先中断出路机和保存该进程的处理机现场，然后被阻塞进程置阻塞状态后插入等待队列中，再转进程调度程序选择新的就绪进程投入运行。

阻塞原语的流程如图 6-c

(2) 进程的唤醒

(a) 进程唤醒的两种方法

1) 由系统进程唤醒

系统统一控制事件的发生并将“事件发生”这一消息通知等待进程。

2) 由事件发生进程唤醒

在这种情况下，事件发生进程和唤醒进程之间是合作关系。

(b) 唤醒原语的流程如图 6-d

6.3.4 进程的调度算法举例

1. 先来先服务 (first come first service, FCFS)

将用户作业和就绪进程按提交顺序转变为就绪状态的先后排成队列，并按照先来先服务的方式进行调度处理，是一种最简单的方法。在没有特殊理由要优先调度某类作业或进程时，从处理的角度看，FCFS 方式是一种最合适的方法，因为无论是追加还是取出一个队列元素在操作上都是最简单的。该算法的优点是实现简单，缺点是对那些执行时间较短的进程来说，将等待较长的时间，从而降低 CPU 的利用率。在实际操作系统中，FCFS 算法常常和其它的算法配合起来使用，例如，基于优先级的调度算法就是对具有同样优先级的进程采用 FCFS 算法。

2. 轮转法 (round robin, RR)

轮转法的基本思路是让每个进程在就绪队列中的等待时间与享受服务的时间成比例。轮转法的基本概念是将 CPU 的处理时间分成固定大小的时间片，例如，几十毫秒到击败毫秒。如果一个进程在被调度选中之后用完了系统规定的时间片，但又未完成要求的任务，则它自行释放自己所占有的 CPU 而排到就绪队列的末尾，等待下一次调度。同时，进程调度程序又去调度当前就绪队列中的第一个进程。显然，轮转法只能用来调度分配那些可以抢占的资源。这可以抢占的资源可以随时剥夺，而且可以将它们再分配给别的进程。CPU 是可抢占的资源的一种。但打印机等是不可抢占的资源。另外，时间片长度的选择是根据系统对响应时间的要求和就绪队列中所允许的最大进程数确定的。

3. 多级反馈轮转法 (round robin with multiple feedback)

(1) 在轮转法中，加入到就绪队列的进程有三种情况：

(a) 分给它的时间片用完，但进程还未完成，回到就绪队列的末尾等待下次调度去继续执行；

(b) 分给该进程的时间片未用完，只是因为请求 I/O 或由于进程的互斥与同步关系而被阻塞，当阻塞解除之后再回到就绪队列；

(c) 新创建进程进入就绪队列。

(2) 如果对这些进程区别对待，给予不同的优先级和时间片，可提高系统资源的利用率。例如，把就绪队列按照进程到达就绪队列的类型和进程被阻塞的原因

分成不同的就绪队列，每个队列按 FCFS 原则排列，不同队列中的进程享有不同的优先级，这样，当一个进程在执行完它的时间片之后，或从睡眠中被唤醒以及被创建之后，将进入不同的就绪队列。

(3) 多级反馈轮转法与优先级法在原理上的区别是：一个进程在它执行结束之前，可能需要反复多次通过反馈循环执行，而不是优先级中的一次执行。

4. 优先数法 (priority)

1. 所谓优先数法是指系统或用户按某种原则为进程指定一个优先级来表示该作业或进程所享有的调度优先权。该算法的核心是确定进程的优先级。

2. 优先级的确定方法

优先级的确定方法有两种，分别为动态法和静态法，

(1) 静态法根据进程的静态特征，在进程开始之前就确定它们的优先级，一旦开始之后就不能改变。

(2) 动态法把进程的静态特征和动态特征结合起来确定作业或进程的优先级，随着进程的执行过程，其优先级不断变化。

3. 静态优先级的确定原则

(1) 作业调度中的优先级确定的原则

(a) 由用户自己根据作业的紧急程度输入一个适当的优先级；

(b) 由系统或操作员根据作业类型指定优先级；

(c) 系统根据作业要求资源情况确定优先级。

(2) 进程调度中的优先级确定的原则

(a) 按进程的类型给予不同的优先级。一般地进程被分为系统进程和用户进程，对于用户进程和系统进程均可以再按照功能分为不同的类型并赋予不同的优先级。

(b) 将作业的优先级作为它所属进程的优先级。

4. 静态优先级的优缺点

基于静态优先级的调度算法的优点是实现简单，系统开销小；缺点是系统的效率底下，调度性能不高。

5. 动态优先级的确定原则

(1) 根据进程占有 CPU 的时间的长短来决定。

(2) 根据就绪进程等待 CPU 的时间长短来决定。

6. 动态优先级的优缺点

优点是调度性能高，系统资源的利用率高，缺点是系统开销大。

6.3.5 进程控制块

1. **进程控制块** (process control block, PCB) 是进程的重要组成部分，是操作系统能“感知”进程存在的唯一标志，它和进程是一一对应的，操作系统正是通过管理 PCB 来管理进程的。

2. PCB 的数据结构

PCB 是记录型的数据结构，其作用主要是描述进程的动态特征。主要内容有：

进程标识号—系统内部用于标识进程的整数；

进程状态—指进程当前所处的状态（就绪、运行、等待）；

进程标志—指是系统进程还是用户进程，程序实体是在内存还是在外存；

进程优先数—它是一个表示进程优先级的整数；

程序地址—指该进程的程序在内存或外存的地址；

现场保留区—在进程交换时保存其程序运行的 CPU 现场；

同步、互斥机构—同步和互斥的信号量；

通信结构—它包含一些指针，这些指针指向相应的通信队列或通信信箱；

家族联系—包含指向父进程和子进程的指针；

资源清单—本进程当前已分得的资源。

6. 4 进程通信

6. 4. 1 进程的同步与互斥

1. 进程同步

(1) 进程同步的定义

进程同步是进程间共同完成一项任务是直接发生相互作用的关系，也就是说，这些具有伙伴关系的进程在执行时间次序上必须遵循确定的规律。

(2) 进程同步的例子

SP00Ling 系统中的输入功能可以由两个进程 A 和 B 完成，进程 A 负责从读卡机上把卡片上的信息读到一个缓冲区中，进程 B 负责把该缓冲区中的信息进行加工并写到外存输入井中。要实现两者的协同工作，两个进程必须满足如下的制约关系：只有当缓冲区的内容取空时，进程 A 才能向其中写入新信息；只有当缓冲区写满时，进程 B 才能从中取出内容作进一步加工和转送工作。可见，在缓冲区内容区空时，今晨 B 不应该继续运行，需要等待进程 A 向其中送入新的信息；反之，当缓冲区中的信息尚未取走时，进程 A 应等待，防止把原有的信息冲掉，造成丢失信息的结果。进程 A 和进程 B 就是一种同步关系。

2. 进程互斥

(1) 进程互斥的定义

一组并发进程中的一个或多个程序段，因共享某一公有资源而导致它们必须以一个不允许交叉执行的单位执行。

进程的互斥是因为对同一物理资源的竞争而产生的相互制约关系。

(2) 进程互斥的例子

两个进程使用一台打印机。

3、同步与互斥特点比较

同步与互斥的特点比较见表 6-4

同 步	互 斥
进程—进程	进程—资源—进程

时间次序上受到某种限制 相互清楚对方的存在及其作用，交换信息 往往指有几个进程共同完成一个任务 举例：生产与消费之间，发送与接受之间，作者与读者之间，供者与用者之间	竞争到某一物理资源时不允许进程工作 不一定清楚其它进程情况 往往指多个任务多个进程间通讯制约 举例：交通十字路口，单轨火车的拨道岔
---	--

表 6-4 同步与互斥特点比较表

6. 4. 2 临界资源和临界区

我们包一次只允许一个进程使用的资源称为临界资源，而把在每个进程中访问临界资源的程序段称为临界区。要进入临界区的若干个进程必须满足如下关系：

- (1) 一次只允许一个进程进入临界区。
- (2) 任何时候，处于临界区的进程不得多于一个。
- (3) 进入临界区的进程要在有限的时间内退出。
- (4) 如果不能进入自己的临界区，则应让出处理机资源。

6. 4. 3 同步、互斥机制的实现及应用

1、用锁操作原语实现互斥

(1) 实现

为共享资源设置一把锁

W=0 表示共享资源（分配表）可用；

W=1 表示共享资源（分配表）不可用，已有一进程访问。

用类 ALGOL 语言编程如下：

加锁原语 LOCK (W)

L: if W=0 then W:=1 else goto L:

（说明：测试和设置指令；循环等待该资源释放）

开锁原语 UNLOCK (W)

W: =0

加锁/开锁原语工作流程如图 6-2

N

Y

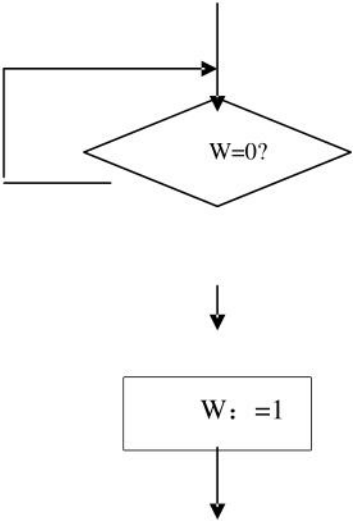


图 6-2 加锁/开锁原语工作流程图

(2) 局限性

只要有一个进程由于执行 LOCK (W) 而进入临界区，则其它进程在检查锁状态时都将反复执行 LOCK (W) 原语，从而导致处理机繁忙。现在一般采用硬件指令来解决互斥进入临界区问题。

2、信号量及 P、V 操作原语

P、V 操作是荷兰科学家 E. W. Dijkstra 在 1965 年提出的一种解决同步、互斥问题的更通用的方法，并在 THE 操作系统中得以实现。P 是荷兰语发信号的开头字母，V 是等待的开头字母。

(1) 信号量

信号量，也叫信号灯，一般是由两个成员组成的数据结构，其中一个成员 S 是整型变量，表示该信号量的值，另一个成员 Q 是指向 PCB 的队列。

<信号量>= (S, Q)

信号量的值与相应资源的使用状态有关。当它的值 >0 时，它表示可用资源的数量；当它的值 <0 时，其绝对值表示等待使用该资源的进程个数，即在该信号量队列上排队的 PCB 的个数。信号量的一般数据结构及 PCB 队列如图 6-e

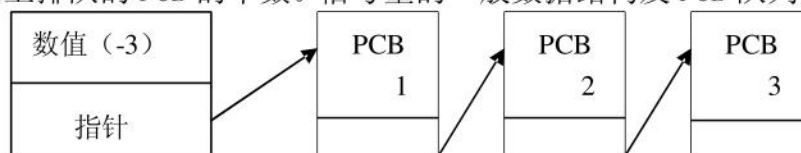


图 6-e 信号量的一般结构及 PCB 队列

(2) P、V 操作

设信号量为 S，对 S 的 P 操作记为 P(S)，对它的 V 操作记为 V(S)。P、V 操作的含义是：

(a) P 操作原语 P(S)

P(S)操作顺序执行下述动作：

- 1) $S=S-1$
- 2) $S>=0$ 说明当前进程 q 有资源可执行；
- 3) $S<=0$ 说明无资源，则当前进程刮起或封锁，将该进程插入排队到该信号量等待队列的队尾，并放弃处理机，进行等待（直到其它进程在 S 上执行 V 操作，把资源释放出来为止）。

流程图见图 6-f

(b) V 操作原语 V(S)

V(S)操作顺序执行下述动作：

- 1) $S=S+1$ ；
- 2) 如果 $S>0$ ，则该进程继续运行；
- 3) 如果 $S<=0$ ，则释放信号队列上的第一个 PCB（即信号量指针所指向的 PCB）所对应的进程（把阻塞态改为就绪态），执行 V 操作的进程继续运行。流程图见图 6-g

3、用 P、V 原语实现进程互斥

用 P、V 原语实现两并发进程 P_A ， P_B 互斥的描述如下：

- 1) 设 sem 为互斥信号量，其取值范围为 (1, 0, -1)。

其中 $\text{sem}=1$ 表示进程 P_A , P_B 都未进入临界区 S , $\text{sem}=0$ 表示进程 P_A 或 P_B 已进入临界区 S , $\text{sem}=-1$ 表示进程 P_A 和 P_B 中, 一个进程已进入临界区, 而另一个进程等待进入临界区。

2) 描述:

P_A :

$P(\text{sem})$

$\langle S \rangle$

$V(\text{sem})$

...

P_B :

$P(\text{sem})$

$\langle S \rangle$

$V(\text{sem})$

...

4、生产者—消费者问题

把并发进程的同步和互斥问题一般化, 可以得到一个抽象的一般模型, 即生产者—消费者问题。计算机系统中, 每个进程都申请使用和释放各种不同类型的资源, 这些资源即可以是象外设、内存及缓冲区等硬件资源, 也包括临界区、数据、例程等软件资源。我们把系统中使用某一类资源的进程称为该资源的消费者, 而把释放同类资源的进程称为该资源的生产者。

设有若干个生产者进程 $P_1, P_2, \dots, P_N$, 若干个消费者进程 $C_1, C_2, \dots, C_M$, 它们通过一个有界缓冲池联系起来。

为了描述生产者—消费者问题, 可以设置若干个信号量 (semaphore):

full , empty 和 mutex 。其中 mutex 是互斥信号量, full , empty 是同步信号量。

编程示意如下:

```
semaphore full = 0;
```

```
semaphore empty = 0;
```

```
semaphore mutex = 1;
```

```
{
```

```
  /*生产者*/
```

```
  P(empty);
```

```
  P(mutex);
```

```
  /*送数据入缓冲区某单元*/
```

```
  V(mutex);
```

```
  V(full);}
```

```
  /*消费者*/
```

```
  P(full);
```

```
  P(mutex);
```

```
  /*取缓冲区中某单元数据*/
```

```
  V(mutex);
```

```
V(empty);}  
}
```

说明：

(1) 它是一个同步问题

(a) 消费者想要接收一个数据时，有界缓冲区中至少有一个单元是满的；

(b) 生产者想要发送一个数据时，有界缓冲区至少有一个是空的。

(2) 它是一个互斥问题

由于有界缓冲区是临界资源，因此，各生产者进程和各消费者进程必须互斥执行。

5、进程通讯

(1) 进程通讯及分类

进程通讯是指进程间的信息交换。进程通讯方式大致分为三类：共享存储器、消息传递和管道。

(2) 消息传递

消息传递方式分为直接通信方式和间接通讯方式两种，而一直接通讯方式的消息缓冲为基本的进程通讯方式。

(3) 消息缓冲

发送进程直接将消息挂在接收进程的消息缓冲队列上，接收进程从消息缓冲队列中得到消息。基本的有发送（send）消息和读取（read）消息两条原语。

(a) send 用于进程发送信息。

例如，甲进程向乙进程发送消息的过程如下：

- 1) 甲进程在发送消息前，在本进程所占空间中开辟一发送区；
- 2) 使用 send 原语 send(A)；
- 3) send 程序向系统申请一个消息缓冲区，将发送区中消息正文、长度和发送进程名填入；
- 4) 将缓冲区挂到接收消息的乙进程消息链链尾；
- 5) 退出 send，甲进程继续执行。

(b) read 用于进程接收消息。

例如，乙进程读甲进程发送的消息的过程如下：

- 1) 乙进程在接收消息前，在本进程所占空间中开辟一接收区；
- 2) 使用 read 原语 read(B)；
- 3) read 程序将乙进程消息链区中第一个消息缓冲区中的内容、长度、本消息发送者名字，填入接收区；
- 4) 将缓冲区从消息链中摘除，释放缓冲区；
- 5) 退出 read 程序，乙进程继续执行。

详见图 6-3 所示。

6.5 死锁

6. 5. 1 死锁的概念

死锁是两个或两个以上的进程中的每一个都在等待其中另一个进程释放资源而被封锁，它们都无法向前推进，这种现象称为死锁。如图 6-h

6. 5. 2 死锁的四个必要条件

1、死锁的起因

死锁的起因是并发进程的资源竞争。产生死锁的根本原因在于系统提供的资源个数少于并发进程所需要的该类资源数。显然，由于资源的有限性，我们不可能为所要求资源的进程无限制地提供资源。但是，我们可以采用适当的资源分配算法，以达到消除死锁的目的。

2、产生死锁的必要条件

(1) 互斥使用：在一段时间内，一个资源只能由一个进程独占使用，若别的进程也要求该资源，则须等待直至其占用者释放；

(2) 保持和等待：允许进程在不释放其已分得的资源的情况下请求并等待分配新的资源；

(3) 非剥夺性：进程所获得的资源在未使用完之前，不能被其它进程强行夺走，而只能由其自行释放；

(4) 循环等待：存在一个等待进程集合，P0 正在等待一个 P1 占用的资源，P1 正在等待一个 P2 占用的资源，…….. Pn 正在等待一个 P0 占用的资源。

3、关于死锁的进一步说明

(1) 死锁是进程之间的一种特殊关系，是由资源竞争引起的僵局关系，因此，当我们提到死锁时，至少涉及到两个进程。虽然单个进程也可能锁住自己，但那是程序设计错误而不是死锁；

(2) 当出现死锁时，首先要弄清楚被锁的是哪些进程，因竞争哪些资源被锁；

(3) 在多数情况下，一系统出现死锁，是指系统内的一些而不是全部进程被锁，它们是因竞争某些而不是全部资源而进入死锁的，若系统的全部进程都被锁住，我们称系统处于瘫痪状态；

(4) 系统瘫痪意味着所有进程都进入了睡眠（阻塞）状态，但所有进程都睡眠了，如果其中至少有一个进程可由 I/O 中断唤醒的话，这并不一定就是瘫痪状态。

6. 5. 3 死锁的表示

1、死锁的表示

死锁可以用有向图来表示，有向图形成环路则形成死锁。例如，有 P1，P2 两个进程，共享一台打印机资源 R1 和一台输入机 R2，在工作使用时，共享资源被独占。死锁的有向示意图见图 6-4

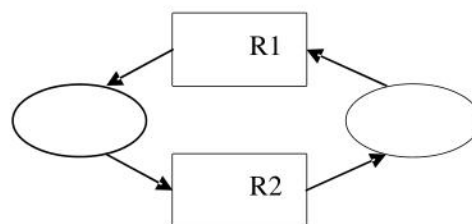


图 6-4 死锁有向示意图

2、死锁定理

- (1) 如果不出现任何环，则此时系统内不存在死锁；
- (2) 如果出现了环，且处于此环中的每类资源均只有一个实体时，则有环就出现死锁，此时，环是系统存在死锁的充分必要条件；
- (3) 如果系统资源分配图中出现了环，但处于此环中的每类资源的个数不全为 1，则环的存在只是产生死锁的必要条件而不是充分条件。

6.5.4 解决死锁问题的基本方法

解决死锁的方法一般可分为预防、避免、检测与恢复等三种。

1、死锁预防

死锁预防的方法主要是打破造成死锁的四个必要条件中的一个，如允许进程同时访问某些资源，然而，这种方法不能解决访问那些不允许被同时访问的资源带来的死锁问题，比如打印机等。另一种方法则是打破资源的部分分配这个死锁产生的必要条件。即预先分配各并发进程所需要的全部资源。

另一种死锁的预防方法是打破死锁的环路条件。即把资源分类按顺序排列，使进程在申请、保持资源时不形成环路。如有 M 中资源，则列出 $R_1 < R_2 < \dots < R_m$ 。若进程 P_i 保持了资源 R_i ，则它只能申请比 R_i 级别更高的资源 R_j ($R_i < R_j$)。释放资源时必须由 R_j 先于 R_i 被释放，从而避免环路的产生。

2、死锁避免

- (1) 静态预先分配所有资源，又称为银行算法。即事先静态说明而后静态分配，破坏部分分配条件，但这种方法的资源利用率低。
- (2) 受控资源分配法：1969 年由 Haberman 提出，分配资源前先检查会不会发生死锁，要求各进程说明所需资源，将资源分类，按不同策略分配，又称为静态说明动态分配。

3、死锁检测与恢复

死锁的检测算法当进程进行资源请求时检查并发进程组是否构成资源的请求和保持环路。有限状态转移图和 petriNet 等技术都可用来有效地判断死锁的发生。死锁的恢复办法很多。最简单的办法是终止各锁住进程，或按一定的顺序中止进程序列，直至已释放到有足够的资源来完成剩下的进程为止，另外，也可以从落锁住进程强迫剥夺资源以解除死锁。

4、处理死锁的综合措施

Howard 在 1973 年提出将解决死锁的基本方法组合起来，并对由不同类资源竞争所引起的死锁采用对它来说是最佳的方法来解决，以此来全面解决死锁问题。这一个思想是基于以下事实：系统内的全部资源可按层次分成若干类，对于每一类，可以使用最适合它的办法来解决死锁问题。由于使用了资源分层技术，在一个死锁环中，通常只包含某一层次的资源，每一个层次可以使用一种基本的方法。因此，整个系统就不会受控于死锁了。

(1) 资源分类

资源可分为如下几类：

- (a) 内部资源：由系统本身使用的资源，例如，进程控制块 PCB；
- (b) 主存储器：主要指用户态程序空间；
- (c) 作业资源：可分配的设备（如磁带机）和文件；
- (d) 交换空间：存放用户作业的后援存储器空间。

(2) 解决死锁的综合措施

对内部资源通过破坏“循环等待”条件，即给资源以线性编序的方法预防死锁；对主存储器通过剥夺资源的办法以防止死锁的产生。因为一个进程总是可以被对换出去的，并且主存储器空间本质上是可以被剥夺的；对作业资源使用死锁避免算法，关于作业要求多少资源的信息可以从作业控制卡中获得；对交换空间可以采用预分配措施。因为存储空间的最大需求量通常是知道的。

6. 6 重点小结

- 1、进程是操作系统中最为重要的概念之一，进程和程序的重要区别是其动态性，进程是程序在并发系统的一次执行，它有一个从产生到消失的生命期。
- 2、进程控制是系统使用一些具有特定功能的程序段（原语）来创建、撤消进程及完成进程各状态（就绪、执行、等待）间的转换，从而达到多进程高效率并发执行和协调、实现资源共享的目的。掌握常用的进程调度算法。
- 3、进程同步和互斥是进程管理的重要内容，掌握同步和互斥的概念，并能利用信号量和 P、V 原语来实现进程间的互斥和同步。
- 4、了解进程通讯的几种方式，掌握消息缓冲的实现方式。
- 5、了解死锁的起因及解决死锁的基本方法。

第七章 操作系统结构和程序设计

7. 1 操作系统的编程概念

1、操作系统编程的发展

在九十年代以前，程序员的精力集中于完成任务的过程，而不是用户与该过程的交互方式，过去评价一个软件的好坏往往是注意源程序的短小精悍和执行的快速性。快速的、清晰的编程是许多程序员努力要达到的理想境界。Windows 操作系统将用户与计算机的交互方式的设计（即人机界面设计）放到了非常重要的位置。同时，Windows 为不同形式的高层次交互提供了相应的机制：应用程序之间、操作系统和应用程序之间、公共的共享代码库和数据库之间。

2、编程语言简史

- (1) 机器语言：以二进制代码“0”、“1”组成的机器指令集合；

- (2) 汇编 (Assembler) 语言：以助记符表示机器指令功能，例如，JOVIAL、forth；
- (3) 高级语言：接近人类语言（英语）和数学语言的计算机语言，例如，BASIC、FORTRAN、Pascal、C、FoxBASE、ORACLE 等；
- (4) 进程 (Process) 概念：例如，UNIX Shell、TCL、PERL 和 Marvel 等；
- (5) 面向对象的语言：例如 C++、Visual BASIC、java 等；
- (6) 新范例计算机语言：例如 ML、Smalltalk、Eiffel 等；

3、不同应用领域的计算机语言

- (1) 科学研究：
例如：FORTRAN、ALGOL、BASIC、APL、Pascal、C、C++、AUTOCAD 等；
- (2) 商业：
例如：COBOL、C、PL/I、4GLs、和 spreadsheet 等；
- (3) 系统：
例如：Assembler、JOVIAL、Forth、C、C++、Ada、java 等；
- (4) 出版：
例如：TeX、Postscript、word、WPS、和 processing 等；
- (5) 人工智能 (AI, artificial intelligence)：
例如：LISP、SNOBOL 和 Prolog 等。

7. 2 结构设计的目标

计算机操作系统一般都有两种运行状态，即用户态 (user mode) 和核心态 (kernel mode)。计算机操作系统的结构设计一般包括内结构和外结构两种结构。传统操作系统内结构是指内部程序模块的层次结构，每一层由若干数量不等的程序模块组成。例如，早期的 UNIX 操作系统版本，如图 7-1 所示。

作业管理
文件管理
设备管理
进程控制和通讯
存储管理
中断处理、进程调度和 P、V 操作等

图 7-1 传统计算机操作系统的内结构（层次结构）

当计算机操作系统内的程序模块运行时，机器处于核心态，其它程序运行时，机器处于用户态，包括操作系统以外的其它系统程序与用户程序一样，都是运行在用户态下而不是运行在核心态下，而可以统称为用户态程序，因而用户程序仅是用户态程序中的一部分。区分用户态和核心态两种运行状态，其目的是使操作系统有一定的特权，使系统运行比较安全可靠。

7. 2. 1 设计目标

- (1) 正确性：实现功能的正确性；
- (2) 安全可靠：系统工作稳定，不容易被破坏，特别是用户的不正常工作状态不致影响系统垮台；
- (3) 可扩充性和可移植性：能适应较大范围工作的运行环境、硬件平台的要求；
- (4) 易维护性：系统具有的自动修补能力强。

7. 2. 2 分层原则

1、层次设计方法从两个不同方向进行：

(1) 自底向上：底层裸机经某种软件层的扩充形成最终目标系统即宿主系统（操作系统），由底层（0 层）扩展到 N 层顶端。例如，早期是先有微机硬件，再设计合适的操作系统。

(2) 自顶向下：已知目标 N 层用户要求，逐级向下进行设计。例如，Windows 的图形控制界面。

2、各层间仅有单向依赖关系，层间独立，结构清晰易懂。一般是有序分层法，例如，上层能调用下层，下层不能调用上层；外层能调用内层，内层不能调用外层，避免产生环路等。其中又可分为：

(1) 半序：允许同一层互相调用。

(2) 全序：不允许同一层内互相调用。例如 THE 系统系统共分为 6 层，如图 7-2 所示。

5	操作员
4	用户程序
3	输入输出管理
2	操作员—进程通讯
1	内存和磁盘管理
0	处理机分配和多道程序

图 7-2 THE 操作系统的分层结构

7. 2. 3 分块原则

1、分块原则

一般按功能分块，形成简单的相对独立模块，所编写的程序模块接口应有比较明确的通讯规则，模块间形成转接、相互依赖甚至循环。

2 进程的观点

用进程的观点进行操作系统设计是继中断概念后的又一重大突破，也就是计算机在执行时不是以一条一条的指令为单元执行，而是以一组特殊指令执行。

3、模块及模块化程序设计

(1) 模块及模块化

模块化是为了获得使用上的灵活性和多样性，是采用标准的可兼容的单元构成系统的一种方法。

(2) 模块化程序设计

模块化程序设计是把一个大的软件系统划分为若干部分，使各部分之间的交往最少，且各自成为具有一定独立功能的程序模块，然后可以对系统中每一部分单独进行设计。

4、模块化程序设计的目的

(1) 使人们确信程序模块的正确性，才能保证所构成的大型软件系统的正确

性；

(2) 各模块尽可能具有较大的独立性，以便能方便地把不同场合下写成的程序模块组合成软件系统。

5、模块的两个属性和概念

(1) 模块程序是相关过程的集合，通常可用树状控制流程来描述；

(2) 每个模块可以由它的输入链路接受数据，并以某种方式变换它，把所得的结果经由它的输出链路发送到其它模块。

7. 2. 4 模块接口法的设计步骤

(1) 按照结构设计的原则，把一个大型软件系统，分成若干个具有一定独立性和大小的模块，每个模块可以解决一部分问题或可实现某种功能，并规定好模块之间的接口。这一步称为系统总体设计。

(2) 进一步明确模块每个模块的内部功能。当模块较大时，同样又可将它进一步细分为若干个子模块。

(3) 当所有的子模块被设计出来后，按照它们的接口关系，把子模块链接在一起组成一个大模块，再链接这些大模块便构成软件系统。

7. 2. 5 模块接口法的优缺点

1、模块接口法的优点

模块接口法设计方法与非结构设计方法比较，有如下一些优点：

- (1) 加速了操作系统的研制过程；
- (2) 促进操作系统的日益完善；
- (3) 增加操作系统的灵活性。

2、模块接口法的缺点

- (1) 很难保证模块设计的正确性；
- (2) 装配的困难性；
- (3) 模块相对独立性差。

7. 3 层次结构设计

7. 3. 1 层次模块化结构设计的必要性

1、结构程序设计的定义

所谓结构程序设计是指为了使程序具有一合理结构，以便于保证和验证其正确性而规定的一套程序设计方法，或者说是“按照一组能增强程序的可阅读性与可维护性的规则而进行程序设计的方法”。用这种方法所设计的程序称为结构化程序。

2、结构程序设计所包含的内容

- (1) 结构码；
- (2) 逐步求精的设计方法；
- (3) 模块接口法；
- (4) 有序分层法；
- (5) 基于进程和线程等的结构设计。

7. 3. 2 操作系统的结构模型

1、层次模型 (layered modularization)

层次模块模型包含模块（相对独立）化和模块间接口定义。系统通常按功能

划分成一些模块，有时模块很小，进一步按由下而上的顺序从硬件层向上组织在若干层次上或自顶向下地组织。

2、整体内核模型 (monolithic kernel)

内核 (功能化的大模块，内部无进程层) 强调操作系统对外的整体性和友好界面。并不追求严格的模块化和层次化。通常内核在处理机的核心状态下运行，具备各种特权，而外层用户程序运行于用户状态。当用户程序请求操作系统服务时，必须通过相应的“访管”或“捕获”指令转入核心态，处理之后再返回用户态。UNIX 内核为无序模块结构，模块间无层次概念，允许任意调用。核心虽无进程概念，但它负责中断处理、进程生灭、进程调度和进程通讯，从而在核心以上提供了进程的运行环境。

3、进程模型 (process-model)

系统的基本模块为进程 (一个独立的控制流)，进程这个概念是对控制流的一种抽象，因此，这种模型具有良好的并发性。除核心以外，操作系统的其它功能都是选择若干进程完成的。

4、对象模型 (object model)

面向对象范型 (object-oriented paradigm) 是以对客观世界自然的抽象为基础设计。对象 (object) 是对客观事物的抽象。每个对象由界面和实体两部分组成，界面规定其引入引出部分，实体定义状态和操作的具体实现。

7. 3. 3 操作系统的结构设计

1、操作系统的结构设计

(1) 总体设计

设计人员要把注意力集中在全局性问题上，即系统的功能设计和结构设计。功能设计是确定系统应具有的功能和实现该功能所采取的策略。以前的操作系统大都采用传统的模块接口法，近年来有序分层法已开始流行起来，最近又提出具有吸引力的、基于线程的结构设计方法。

(2) 逐步求精

在确定操作系统的结构后，便可采用逐步求精的结构程序设计方法进行程序设计。这方法要求从每个问题的全局出发，逐步精细化、具体化。直到成为一可在机器上执行的程序。具体地说，首先把程序在逻辑上分解为若干子结构，对子结构进行描述，并确定相应的数据基，然后再把子结构进行分为更小的结构，对它进行更细微的描述。该过程一直继续下去，直至构成我们所需要的程序。

(3) 结构码

在逐步求精时，应采用单一入口和单一出口的结构码来构成程序。事实上，只要有下述三种类型的结构码，即可构成任何程序的全部流程图。

(a) 序列结构码

它是完全由序列语句串在一起构成的，一个复杂的序列结构可分解为多个较简单的序列结构，如：当过程 A 结束后，一定执行过程 B。

(b) 条件结构码 (if—then—else 和 if—then)

它是由一个判定语句和两个序列语句构成，其中“条件”是布尔表达式。

(c) 重复性结构码 (while—do; repeat—until 等)

用于循环控制

2、有序分层法的类型

有序分层法是在模块接口法的基础上，将模块安置在高低不同的层次中，自底向上或子顶向下进行设计和调试，从而把模块间复杂的依赖关系改为单向依赖关系，把设计决定的无序性该为有序性，从而使设计具有可靠的基础。有序分层结构可分为三种类型：

- (1) 线性有序分层结构；
- (2) 部分有序分层结构；
- (3) 树状分层结构。

3、操作系统的分层原则

- (1) 按程序的嵌套情况排成有序的层次；
- (2) 把活动频繁的模块和为很多模块所公用的模块放在最低软件层，存储器管理模块设置在较低软件层；
- (3) 为获得可移植性和可适应性而设置的软件层，应分别放在较低的层次和最外层。

4、高层调用底层软件的方法

在严格的层次结构中，只允许高层调用底层中的软件，决不允许底层调用高层的软件。在高层调用底层软件的方法有：

- (1) 高层仅能调用它的下一层中的软件；
- (2) 高层能调用所有底层的软件；
- (3) 高层能部分地调用底层中的软件。

7. 6 微内核结构简介

1、微内核（micro-kernel）结构的概念

在层次化结构中，大部分的功能在“内核”中执行。随着功能的扩展，内核逐渐扩大，层次间调用设计大大影响了操作系统设计，功能的扩充、移植、规模的伸缩和安全性等方面出现很多困难。因此产生了微内核结构，只把最基本的操作系统核心功能放在内核中，而把不是最基本功能的服务放在微内核之上，在用户模式中执行。层次内核和微内核的结构如图 7-3

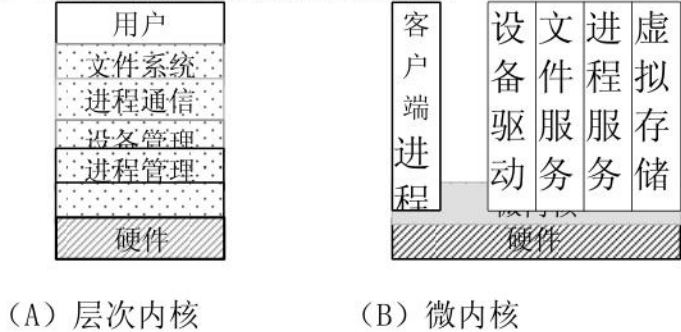


图 7-3 内核结构示意图

2、微内核结构的特点

微内核技术在概念、结构、功能和实现等方面都与以往的操作系统实现有很大的区别。引入了许多成熟的技术，例如：客户/服务器（client/server）结构、面向对象技术、对称多处理（symmetric multiprocessing, SMP）等新技术日益实用化。一般地，内核分为微内核与服务器两个层次，微内核提供各种操作系统的公共基础，由服务器提供各种操作系统的子系统以及用户界面。

3、微内核的基本功能

微内核的基本功能又称为本质功能，这些基本功能包括内进程通讯（inter-process communication, IPC）、虚拟存储（virtual memory, VM）、任务（tasks）、线程（threads）管理和中断（interrupt）处理等。

4、微内核结构主要优点

- (1) 统一的接口：在用户态和核心态（微内核）之间无需进程识别；
- (2) 可伸缩性好：能适应硬件更新和应用变化；
- (3) 可移植性好：所有与具体机器特征相关的代码，全部隔离在微内核中，如果操作系统要移植到不同的硬件平台上，只虚、需修改微内核中极少代码即可；
- (4) 实时性好：微内核可以方便地支持实时处理；
- (5) 安全可靠性强：微内核将安全性作为系统内特性来进行设计，对外仅使用少量的应用编程接口（APIs）；
- (6) 支持分布式系统，支持多处理器的体系结构和高度并行的应用程序；
- (7) 真正面向对象的操作系统。

5、进程与线程的比较

进程与线程的比较见表 7-1。

进 程	线 程
传统的每个进程有其一个地址空间和单一的控制线程，能在其上进行程序的一个操作（执行过程） 为了更好地刻划和实现系统的并发性和并行性 在具有线程机制的系统中，进程是资源分配单位（包括内存空降和代码段）	在分布式系统中，一个进程可拥有多个控制线程 能使同一程序有几个并行执行的路径。 在具有线程机制的系统中，线程是执行单位 由进程派生的执行过程，共享进程的内存和代码段

表 7-1 进程与线程对照表

7. 8 重点小结

- 1、操作系统的内结构和外结构、用户态和核心态运行的特点：给系统以特殊权限，以利于计算机操作系统的安全可靠。
- 2、操作系统的编程特点：模块化结构、层次化结构、内核结构和有序结构。
- 3、微内核结构的主要特点：只把最基本的操作系统核心功能放在内核中，而把不是最基本功能的服务放在微内核之上，在用户模式中执行。